

计算机视觉与深度学习

9-11、全连接神经网络

孙其博 副教授
北京邮电大学 计算机学院

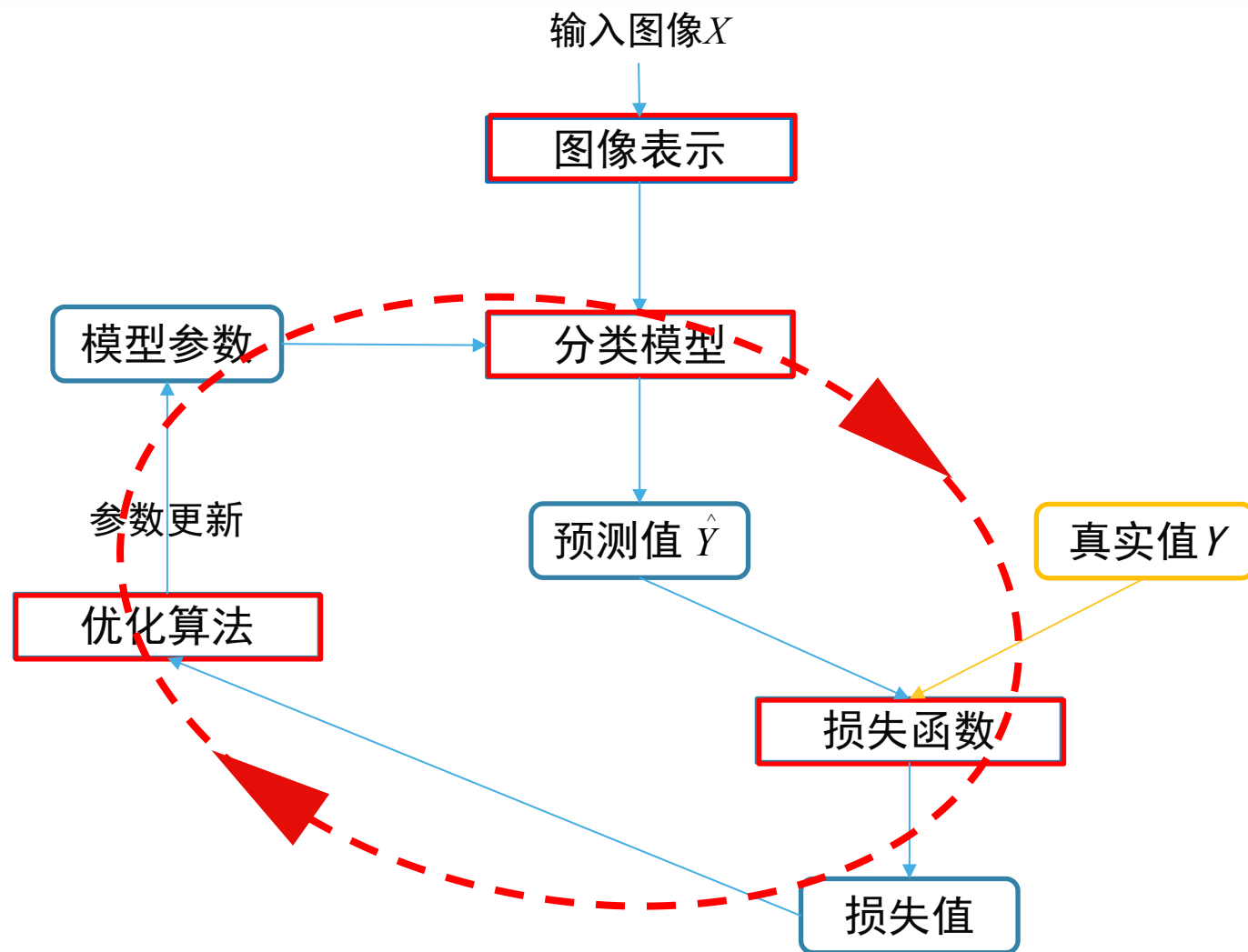


数据驱动的图片分类方法回顾

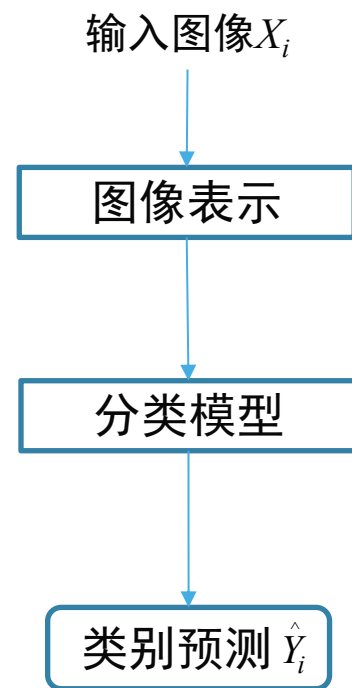
1. 数据集构建
2. 分类器设计与学习
3. 分类器决策



分类器设计与学习



决策



线性分类器设计与学习回顾

输入图像 X

图像表示

基于像素的图像表示

训练过程

- 1、线性分类器定义
- 2、线性分类器的权值
- 3、线性分类器的分界面

- 9、数据集划分
- 10、数据项处理

模型参数

分类模型

参数更新

优化算法

预测值 $\hat{Y}$

真实值 Y

损失函数

损失值

- 4、损失函数定义
- 5、多类支撑向量机损失
- 6、正则项与超参数

- 7、什么是优化？
- 8、梯度下降算法、随机梯度下降算法及小批量梯度下降算法



全连接神经网络设计与学习

输入图像 X

图像表示

图像表示

- 1、多层感知器
- 2、激活函数

模型参数

分类模型

预测值 $\hat{Y}$

真实值 Y

优化算法

参数更新

损失函数

3、SOFTMAX与交叉熵

4、对比多类支撑向量机损失

损失值

训练过程

8、权值初始化

9、批归一化

10、欠拟合、过拟合与Dropout

11、模型正则与超参数调优

5、计算图与反向传播

6、再谈激活函数

7、动量法与自适应梯度

如何表示图像？

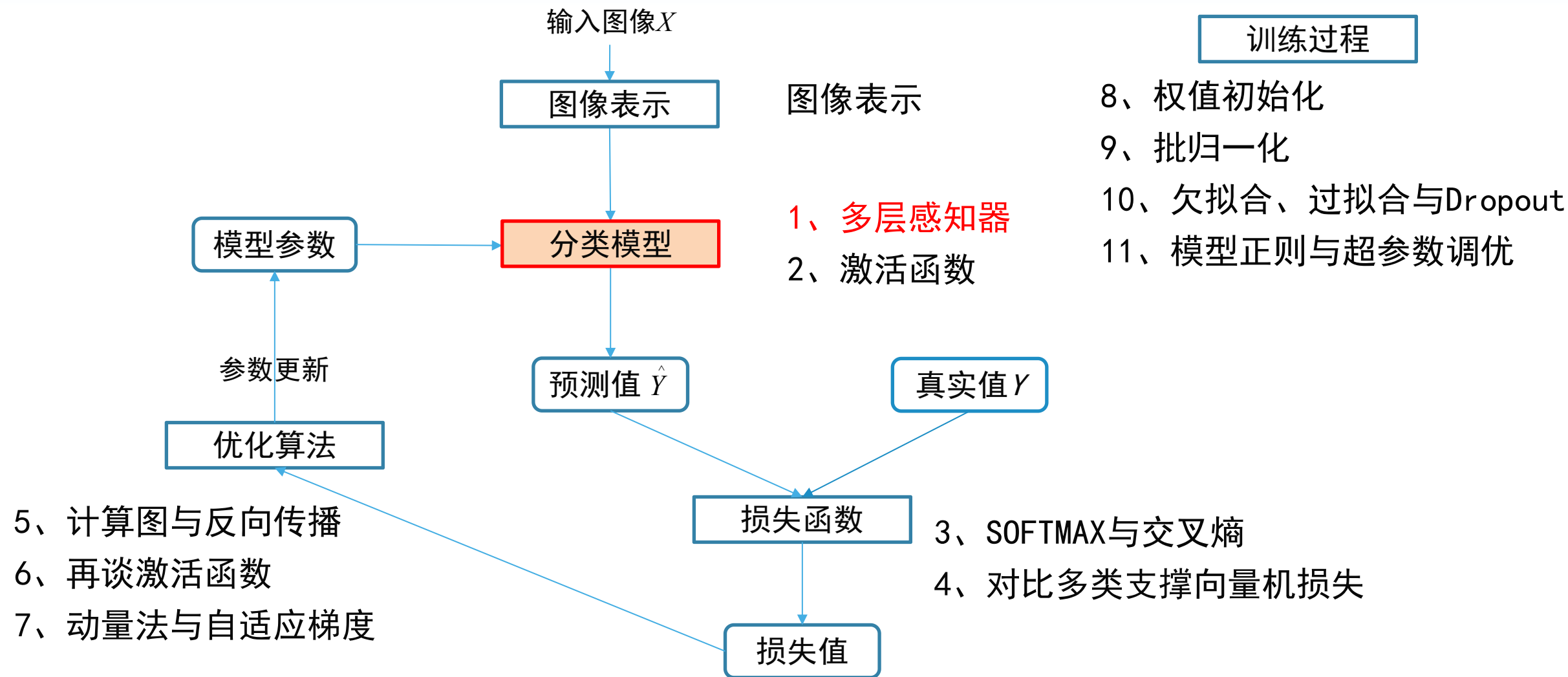
❖ 直接利用原始像素作为特征，展开为列向量。

$$\begin{matrix} \begin{bmatrix} 76 & 32 \\ 4 & 157 \end{bmatrix} \\ \text{图像} \end{matrix} \xrightarrow{\text{将矩阵转成列向量}} x = \begin{bmatrix} 76 \\ 32 \\ 4 \\ 157 \end{bmatrix}$$

CIFAR10中每个图像可表示为一个3072（32*32*3）维的向量



全连接神经网络设计与学习



线性分类器

$$f(x, W) = Wx + b$$

其中， x 代表输入图像，其维度为 d ；

f 为分数向量，其维度等于类别个数 c ，

$W = [w_1 \dots w_c]^T$ 为权值矩阵， $w_i = [w_{i1} \dots w_{id}]^T$ 为第 i 个类别的权值向量，

$b = [b_1 \dots b_c]^T$ 为偏置向量， b_i 为第 i 个类别的偏置。



全连接神经网络

全连接神经网络级联多个变换来实现输入到输出的映射。

两层全连接网络： $f = W_2 \max(0, W_1 x + b_1) + b_2$

三层全连接网络： $f = W_3 \max(0, W_2 \max(0, W_1 x + b_1) + b_2) + b_3$

注意：非线性操作是不可以去掉的。



全连接神经网络的权值

线性分类器: $f(x, W) = Wx + b$

权值模板



两层全连接网络:

$$f = W_2 \max(0, W_1 x + b_1) + b_2$$

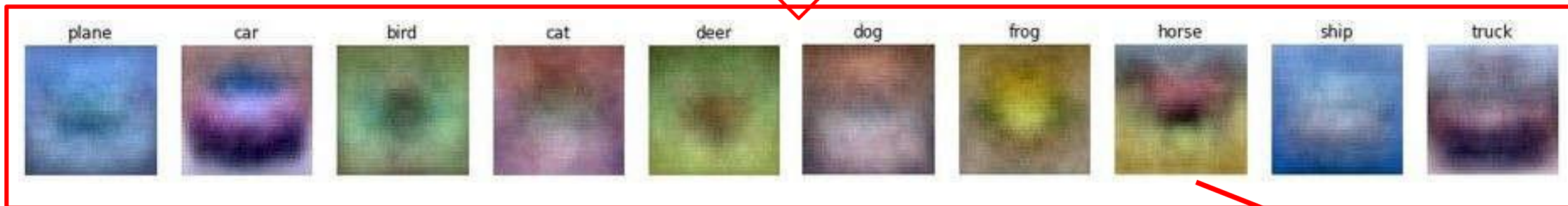
- 线性分类器中的W可看作模板，模板个数由类别个数决定
- 全连接神经网络中：
 - ✓ W_1 也可看作模板；模板个数人为指定
 - ✓ W_2 融合多个模板的匹配结果来实现最终类别打分



全连接神经网络的权值

线性分类器: $f(x, W) = Wx + b$

权值模板



两层全连接网络:

$$f = W_2 \max(0, W_1 x + b_1) + b_2$$

全联接神经网络的描述能力更强。因为调整 W_1 行数等于增加模板个数, 分类器有机会学到两个不同方向的马的模板。



两个马头



全连接神经网络与线性不可分

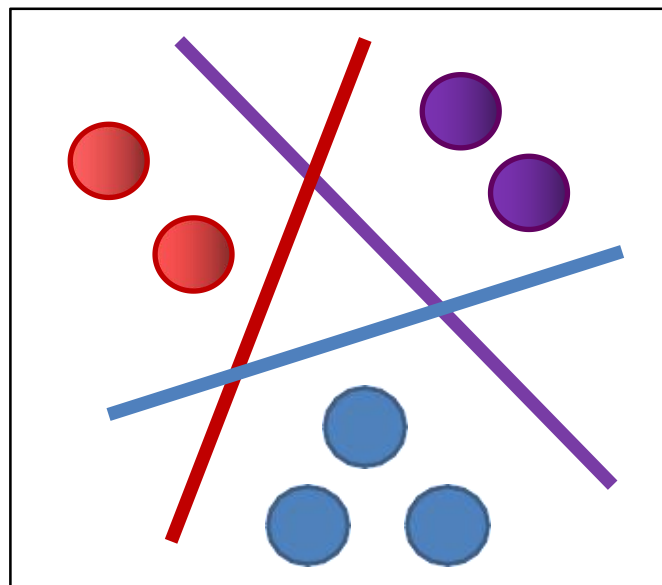
两层全连接网络: $f = W_2 \max(0, W_1 x + b_1) + b_2$

线性分类器:

$$f(x, W) = Wx + b$$

线性可分——至少存在一个线性分界面
能把两类样本没有错误的分开。

线性可分

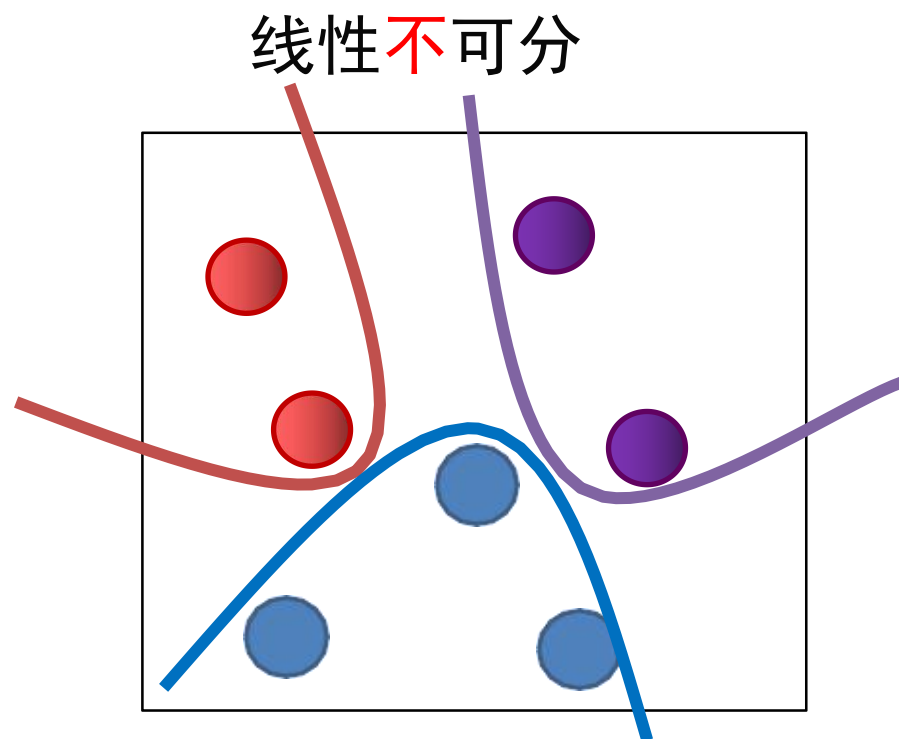


全连接神经网络与线性不可分

两层全连接网络: $f = W_2 \max(0, W_1 x + b_1) + b_2$

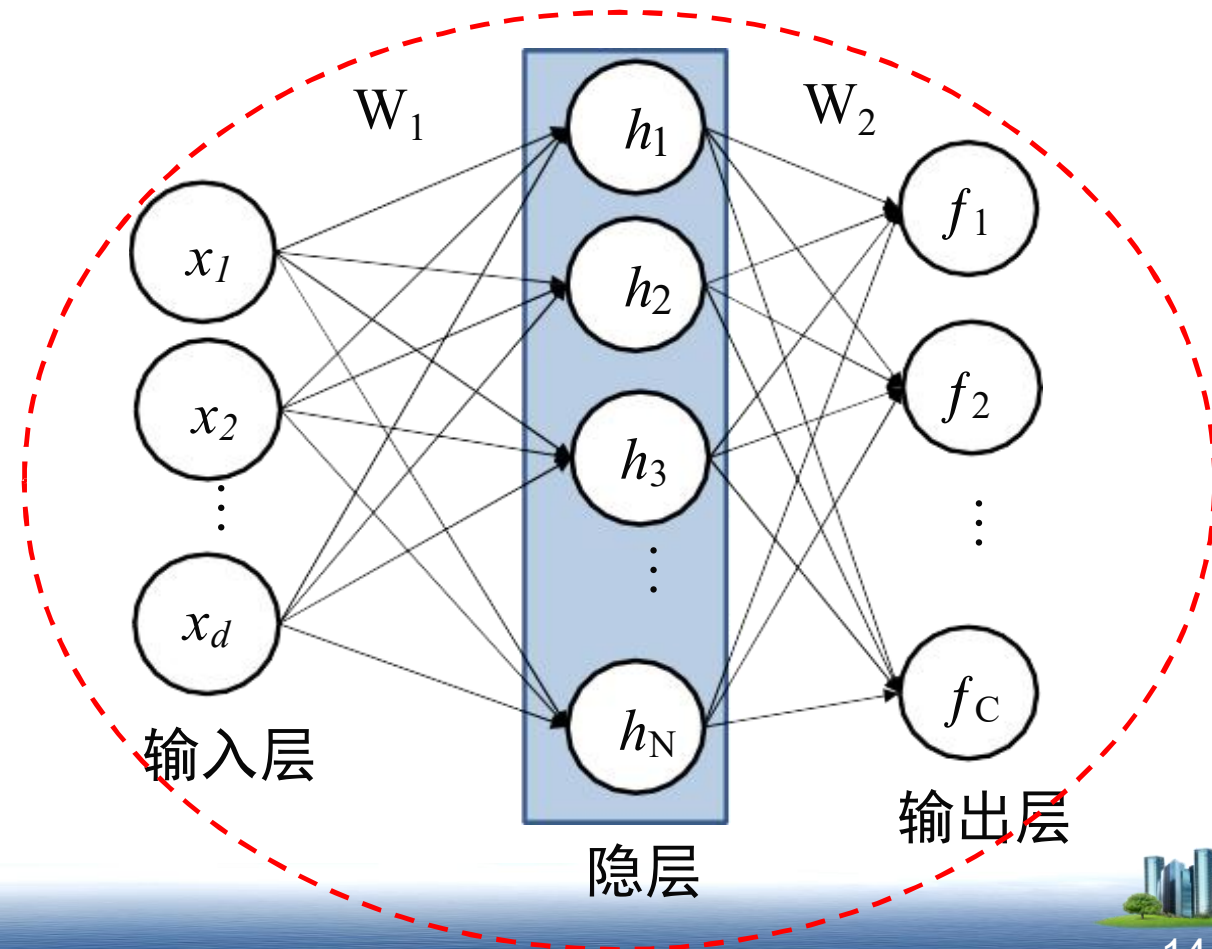
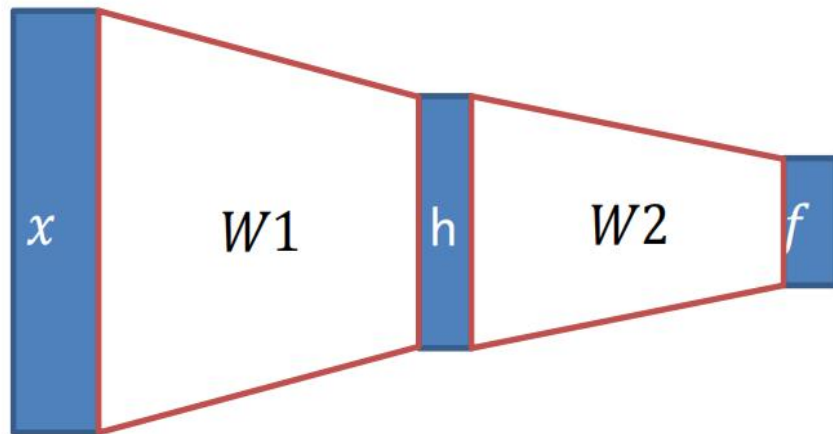
线性分类器: $f(x, W) = Wx + b$

线性可分——至少存在一个线性分
界面能把两类样本没有错误的分开



全连接神经网络绘制与命名

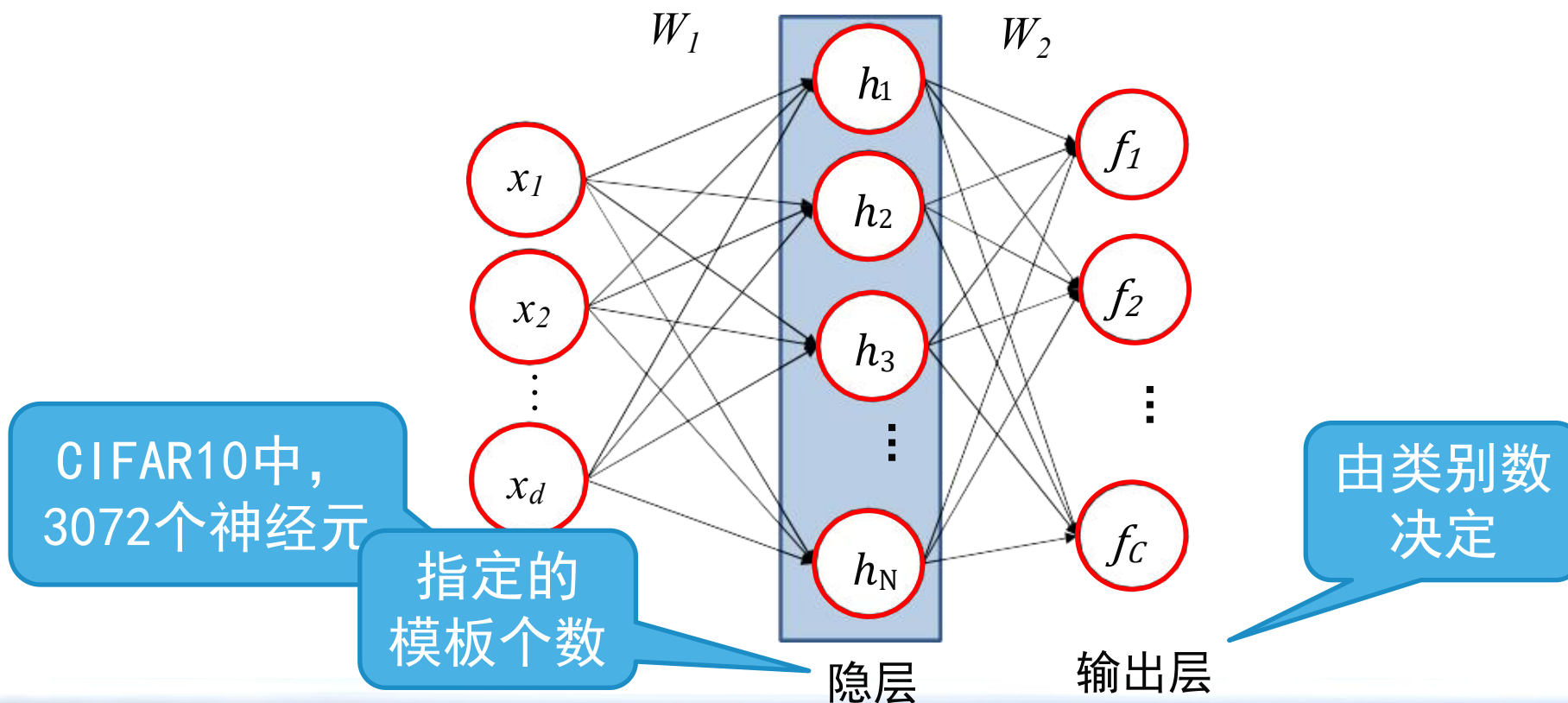
两层全连接网络: $f = W_2 \max(0, W_1 x + b_1) + b_2$



全连接神经网络绘制与命名

两层全连接网络:

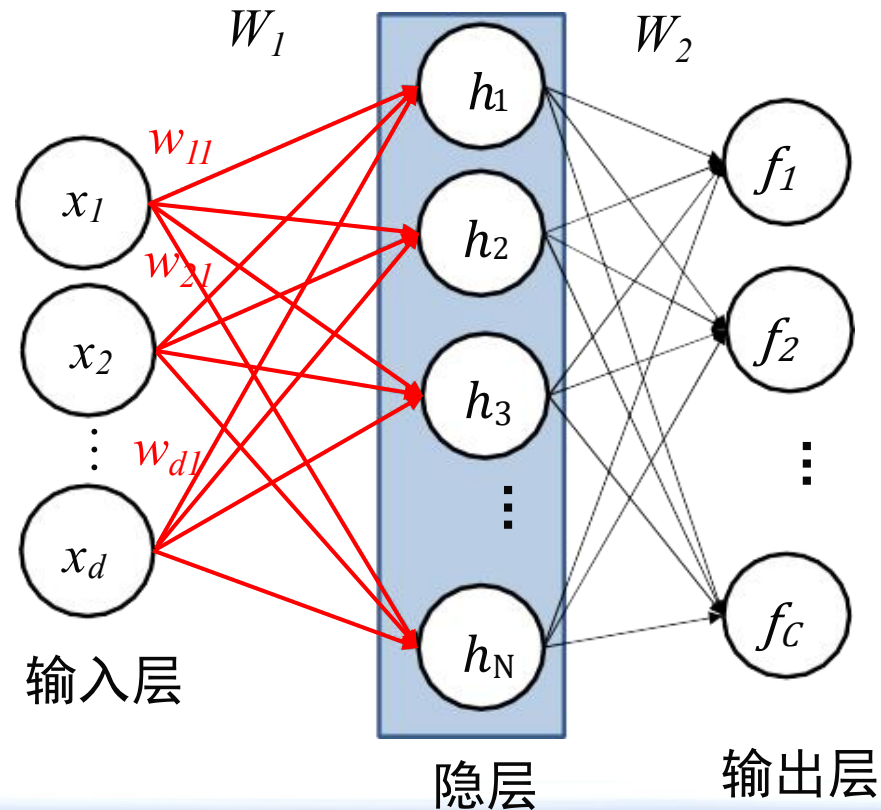
$$f = W_2 \max(0, W_1 x + b_1) + b_2$$



全连接神经网络绘制与命名

两层全连接网络：

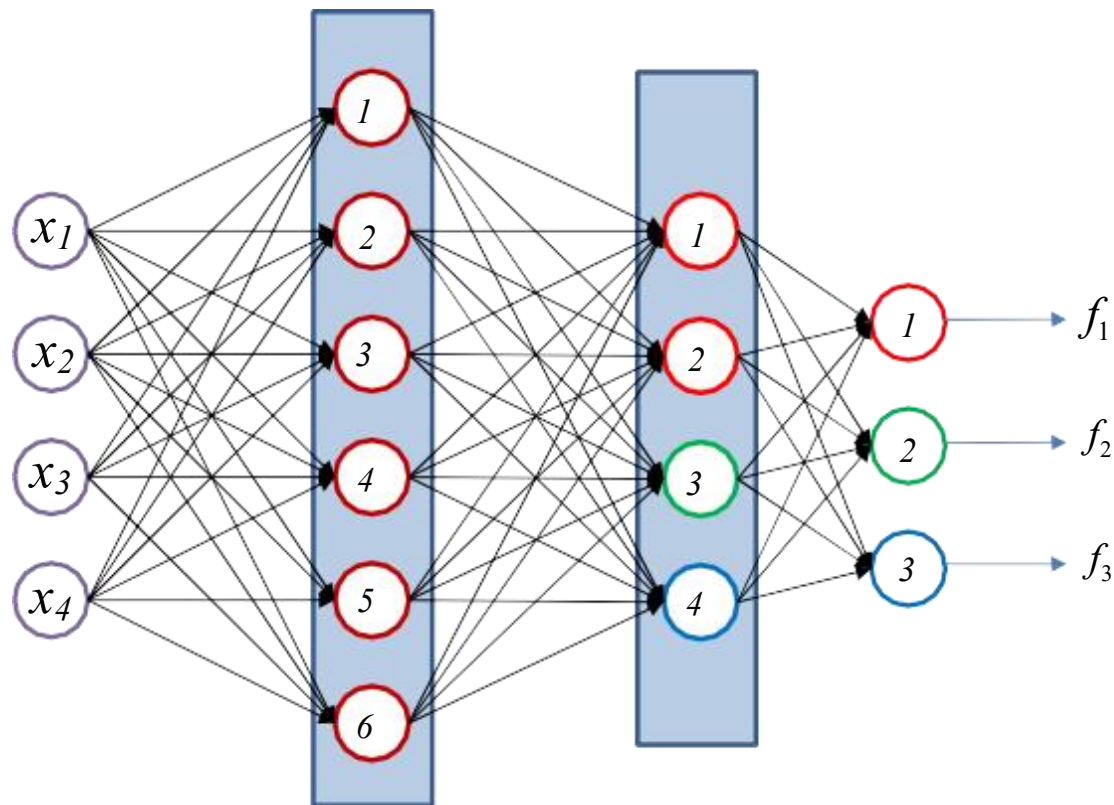
$$f = W_2 \max(0, W_1 x + b_1) + b_2$$



全连接神经网络绘制与命名

N层全连接神经网络——除输入层之外其他层的数量为N的网络

N个隐层的全连接神经网络——网络隐层的数量为N的网络

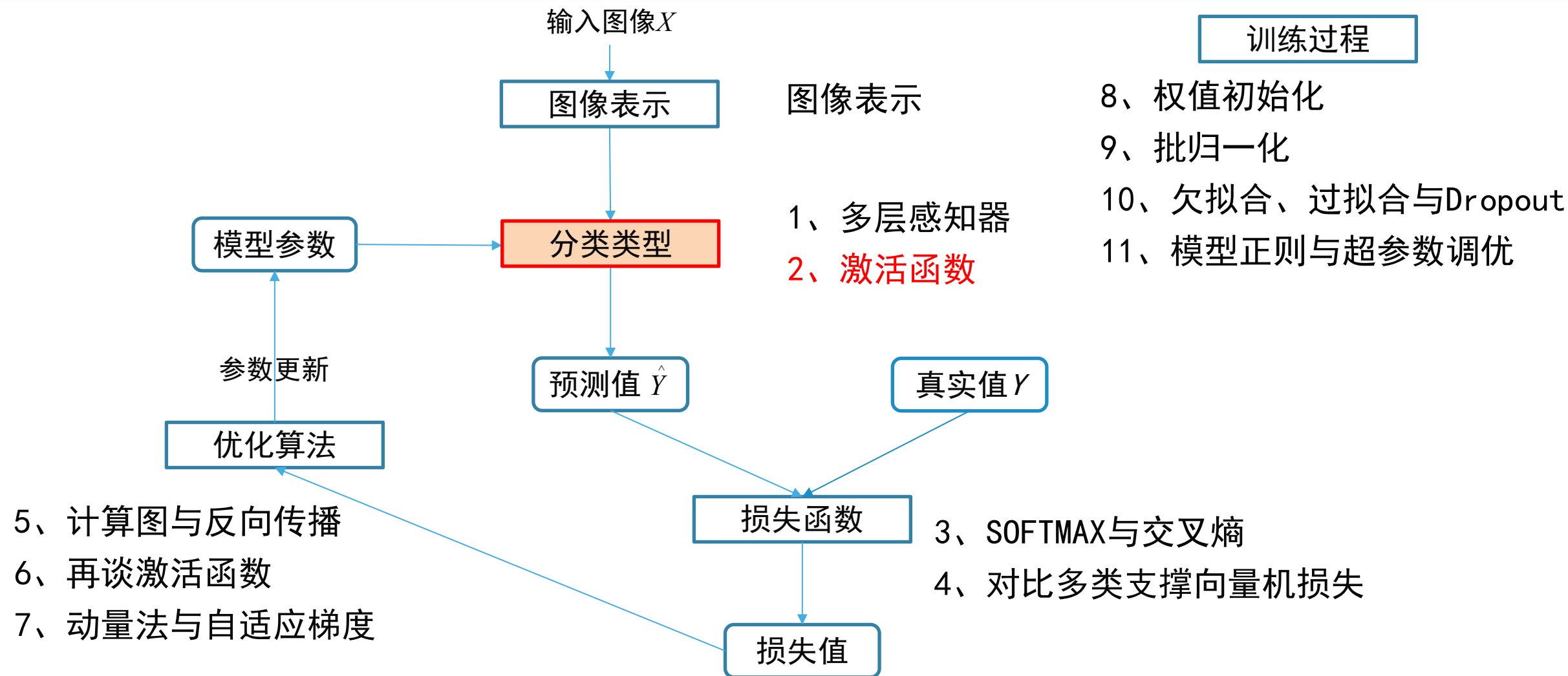


$$f = W_3 \max(0, W_2 \max(0, W_1 x + b_1) + b_2)$$

“3层神经网络” 或者 “2隐层神经网络”



全连接神经网络设计与学习



激活函数

❖ 为什么需要非线性操作？

激活函数

- 三层全连接网络 $f = W_3 \max(0, W_2 \max(0, W_1 x + b_1) + b_2) + b_3$

答：如果网络中缺少了激活函数，全连接神经网络将变成一个线性分类器。

去掉激活函数

$$\begin{aligned} f &= W_3(W_2(W_1 x + b_1) + b_2) + b_3 \\ &= W_3 W_2 W_1 x + (W_3 W_2 b_1 + W_3 b_2) + b_3 \\ &= W' x + b' \end{aligned}$$

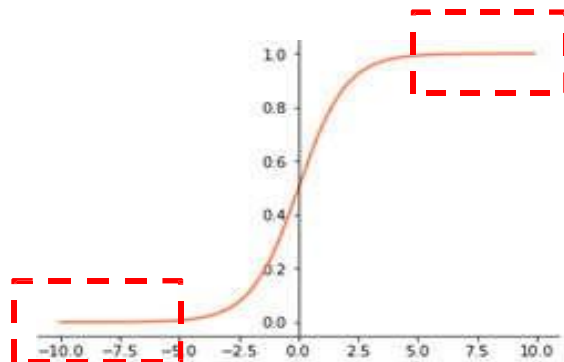


常用的激活函数



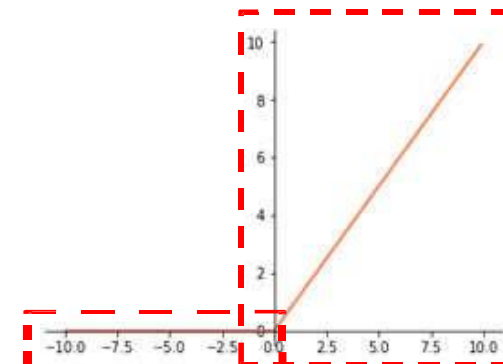
Sigmoid

$$\frac{1}{1 + e^{-s}}$$



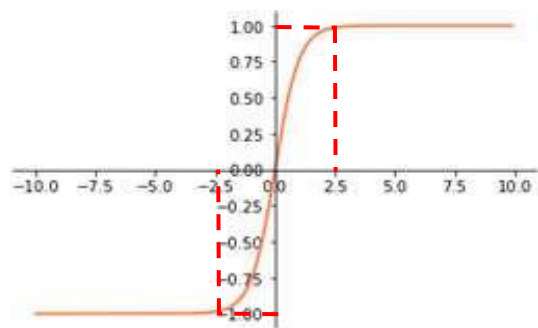
ReLU

$$\max(0, x)$$



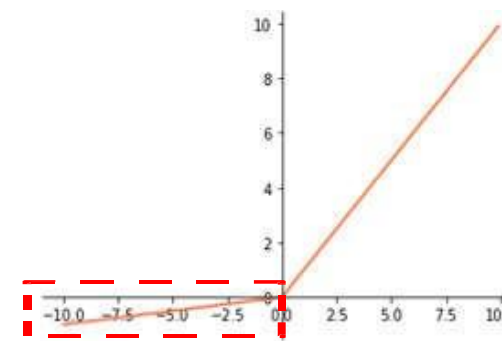
tanh

$$\frac{e^s - e^{-s}}{e^s + e^{-s}}$$



Leaky ReLU

$$\max(0.1x, x)$$



网络结构设计

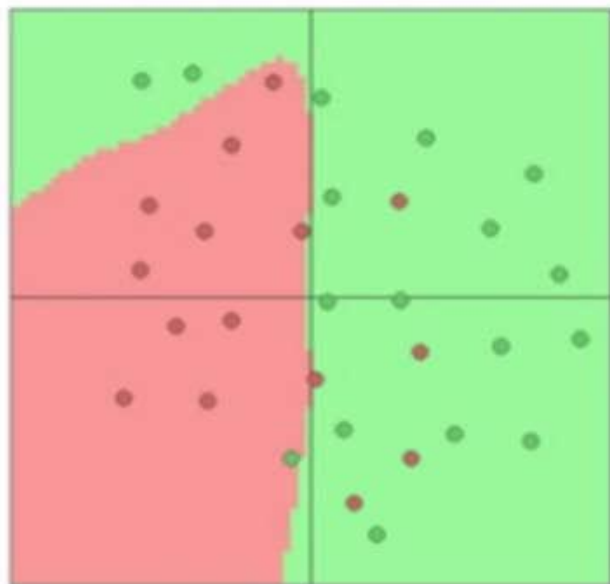


1. 用不用隐层，用一个还是用几个隐层？（深度设计）
2. 每隐层设置多少个神经元比较合适？（宽度设计）

没有统一的答案！



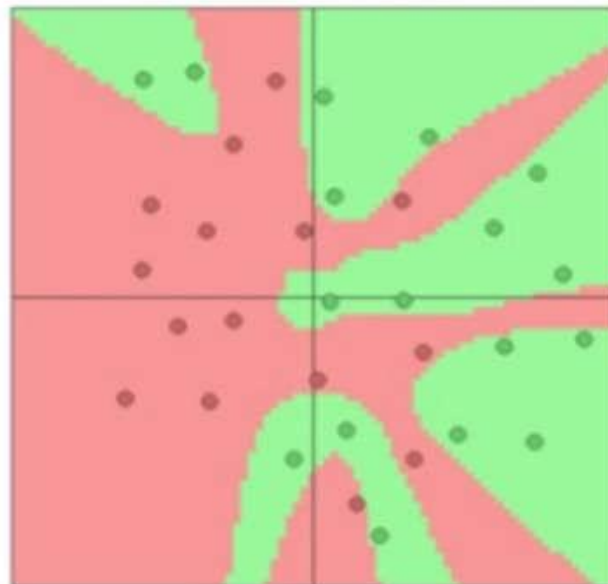
网络结构设计



3个隐层神经元



6个隐层神经元



20个隐层神经元

结论：神经元个数越多，分界面就可以越复杂，在这个集合上的分类能力就越强。



网络结构设计



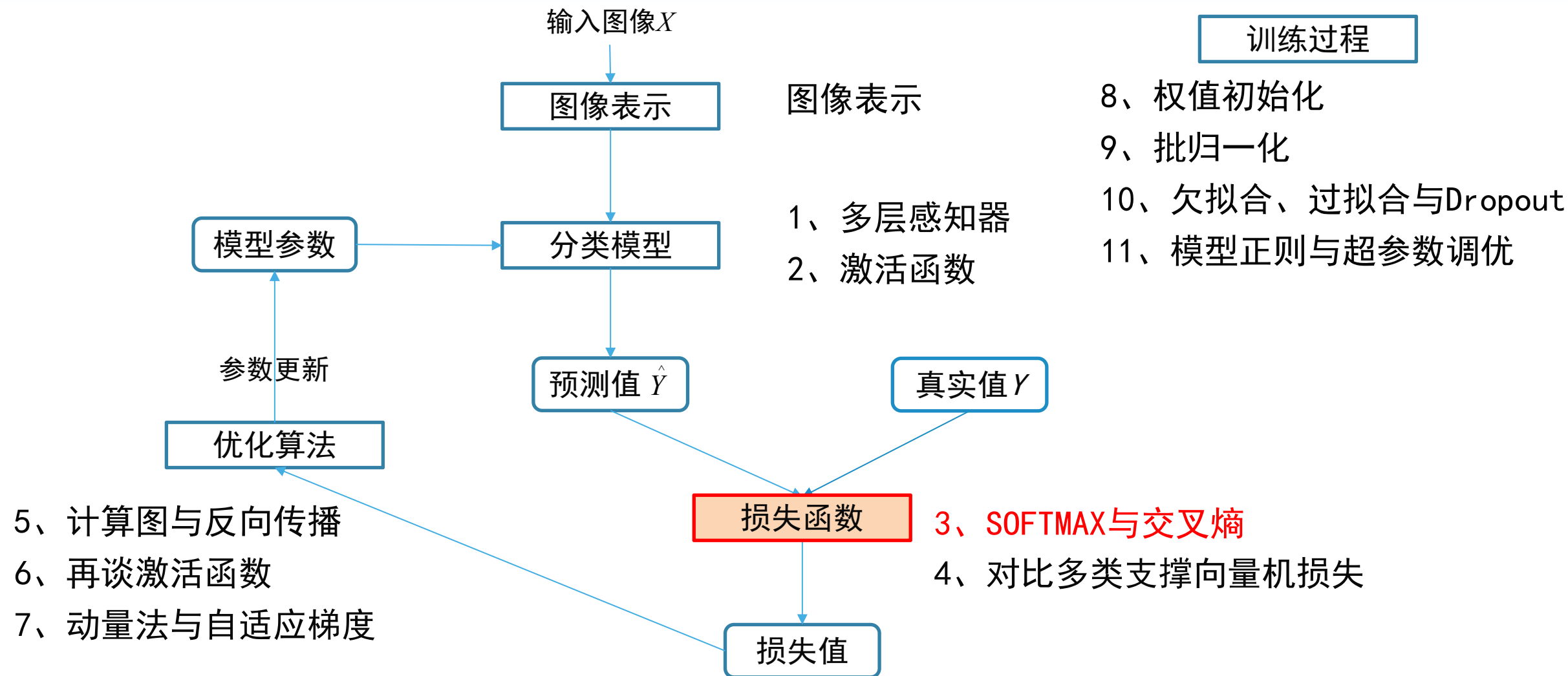
依据分类任务的难易程度来调整神经网络模型的复杂程度。分类任务越难，我们设计的神经网络结构就应该越深、越宽。但是，需要注意的是对训练集分类精度最高的全连接神经网络模型，在真实场景下识别性能未必是最好的（过拟合）。



全连接神经网络小结

- ❖ 全连接神经网络组成：一个输入层、一个输出层及多个隐层；
- ❖ 输入层与输出层的神经元个数由任务决定，而隐层数量以及每个隐层的神经元个数需要人为指定；
- ❖ 激活函数是全连接神经网络中的一个重要部分，缺少了激活函数，全连接神经网络将退化为线性分类器。

全连接神经网络设计与学习

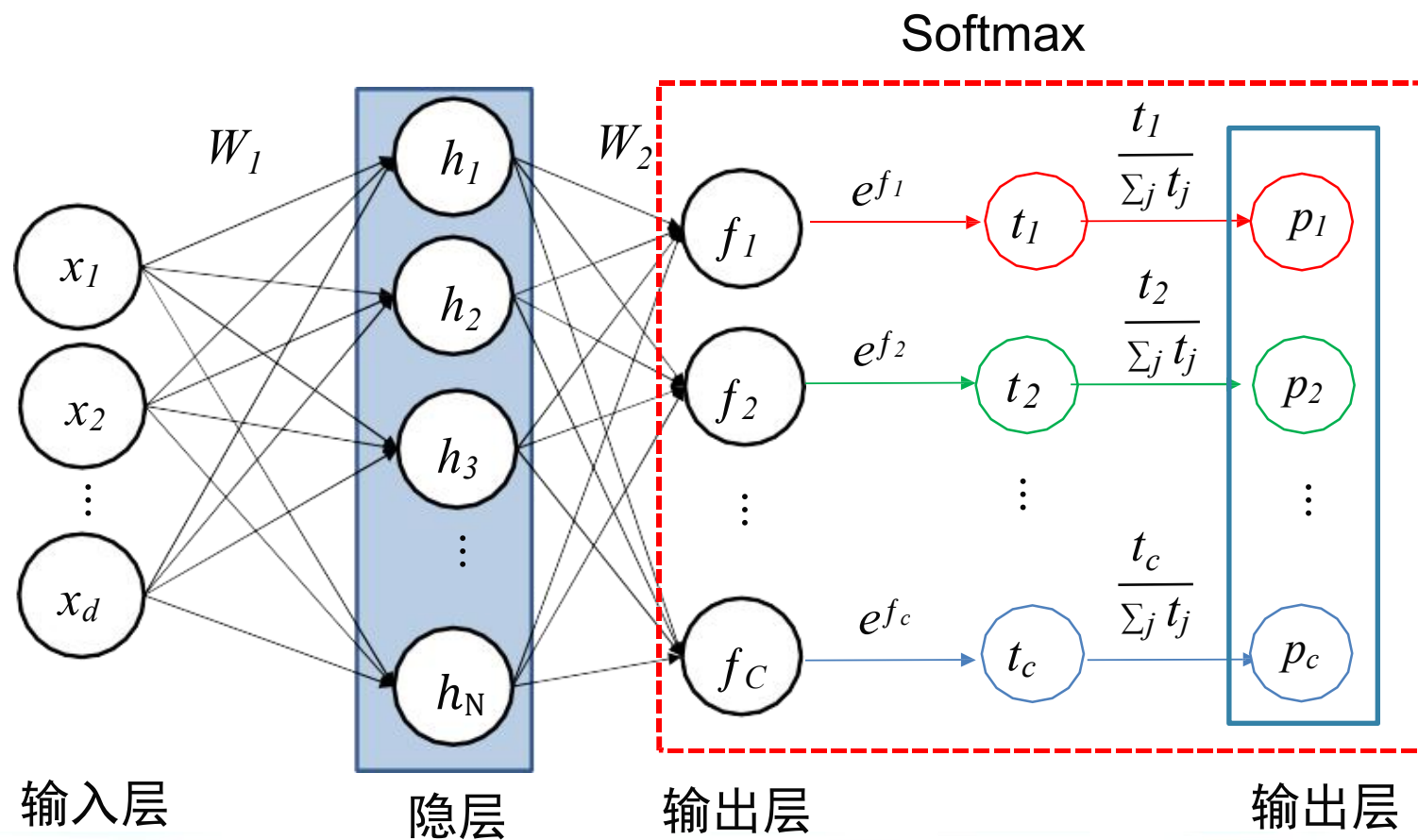


SOFTMAX



两层全连接网络:

$$f = W_2 \max(0, W_1 x + b_1) + b_2$$



SOFTMAX



示例：



图像

$$f = W_2 \max(0, W_1 x + b_1) + b_2$$

0.6
-2.3
1.9

分数

指数

e^{f_i}
1.81
0.10
6.69

归一化

$\frac{t_i}{\sum_j t_j}$
0.21
0.01
0.78

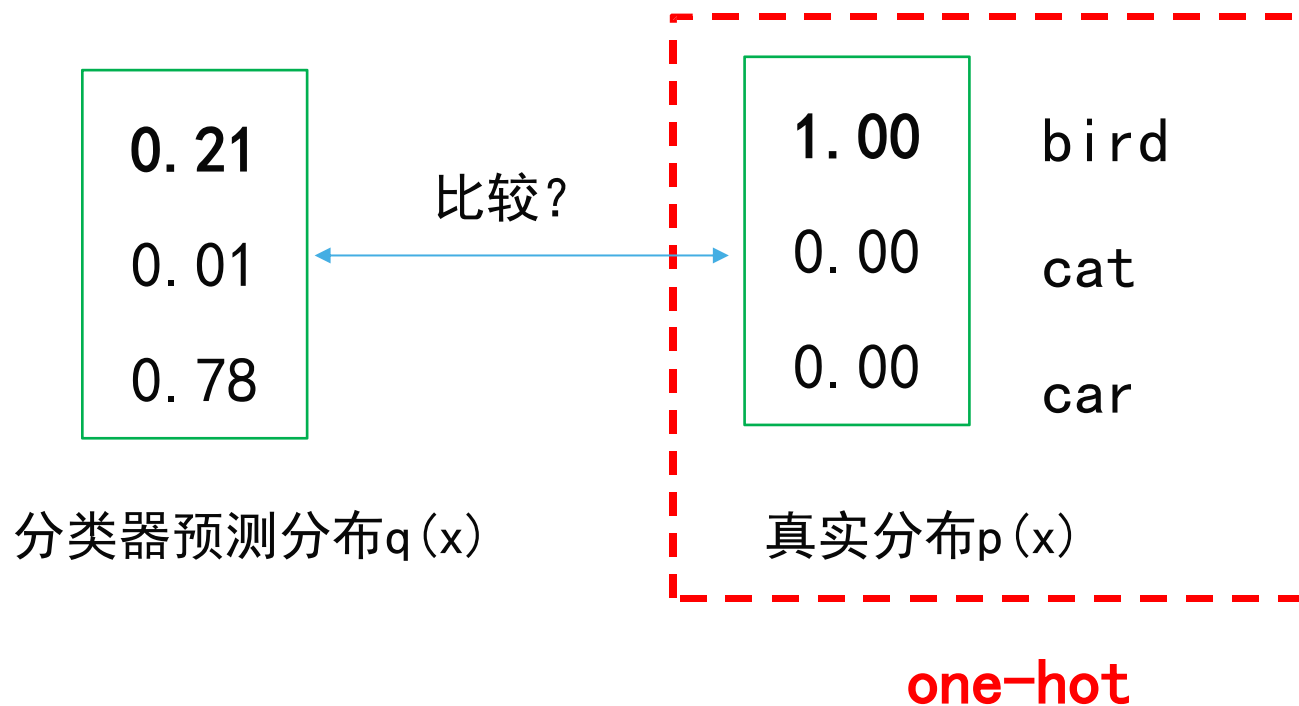
概率

bird
cat
car



交叉熵损失

❖ 如何度量现在的分类器输出与预测值之间的距离？



交叉熵



熵: $H(p) = -\sum_x p(x) \log p(x)$

交叉熵: $H(p, q) = -\sum_x p(x) \log q(x)$

相对熵: $KL(p \parallel q) = -\sum_x p(x) \log \frac{q(x)}{p(x)}$

相对熵 (relative entropy) 也叫KL散度 (KL divergence);
用来度量两个分布之间的不相似性 (dissimilarity) 。

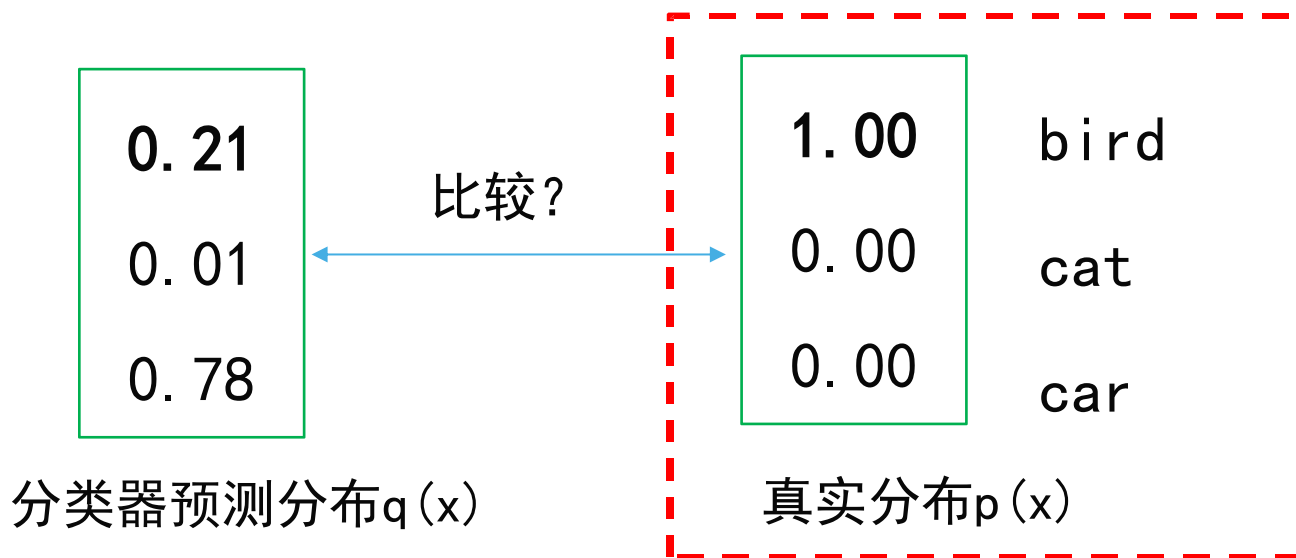
三者之间关系:

$$\begin{aligned} H(p, q) &= -\sum_x p(x) \log q(x) \\ &= -\sum_x p(x) \log p(x) - \sum_x p(x) \log \frac{q(x)}{p(x)} \\ &= H(p) + KL(p \parallel q) \end{aligned}$$



交叉熵损失

❖ 如何度量现在的分类器输出与预测值之间的距离？



注：真实分布为one-hot形式时
交叉熵损失简化为：

$L_i = -\log(q_j)$, 其中j为真实类别

如何衡量两个随机分布的差异？

交叉熵损失：

$$L_i = -\sum_{i=1}^c p(x_i) \log(q(x_i))$$

交叉熵： $H(p, q) = -\sum_{i=1}^c p(x_i) \log(q(x_i))$

全连接神经网络设计与学习



输入图像 X

图像表示

图像表示

分类模型

- 1、多层感知器
- 2、激活函数

模型参数

预测值 $\hat{Y}$

真实值 Y

优化算法

损失函数

3、SOFTMAX与交叉熵

4、对比多类支撑向量机损失

损失值

训练过程

8、权值初始化

9、批归一化

10、欠拟合、过拟合与Dropout

11、模型正则与超参数调优

5、计算图与反向传播

6、再谈激活函数

7、动量法与自适应梯度

参数更新



交叉熵损失 vs 多类支撑向量机损失

多类支撑向量机损失

$$L_i = \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

$$= \max(0, -2.3 - 0.6 + 1) + \max(0, 1.9 - 0.6 + 1)$$

$$= 2.3$$



图像

分类器

0.6
-2.3
1.9

分数 f

e^{f^1}

1.81
0.10
6.69

指数

归一化

$\frac{t_i}{\sum_j t_j}$

0.21
0.01
0.78

概率

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

$$= 0.92$$

交叉熵损失



交叉熵损失 vs 多类支撑向量机损失

假设分数 f
 $y_i=0$

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

[10, -2, 3]

[10, 9, 9]

[10, -100, -100]

问： 相同分数下两种分类器的损失有什么区别？



交叉熵损失 vs 多类支撑向量机损失

假设分数 f
 $y_i=0$

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

[10, -2, 3]

0.0004

0

[10, 9, 9]

0.2395

0

[10, -100, -100]

1.5×10^{-48}

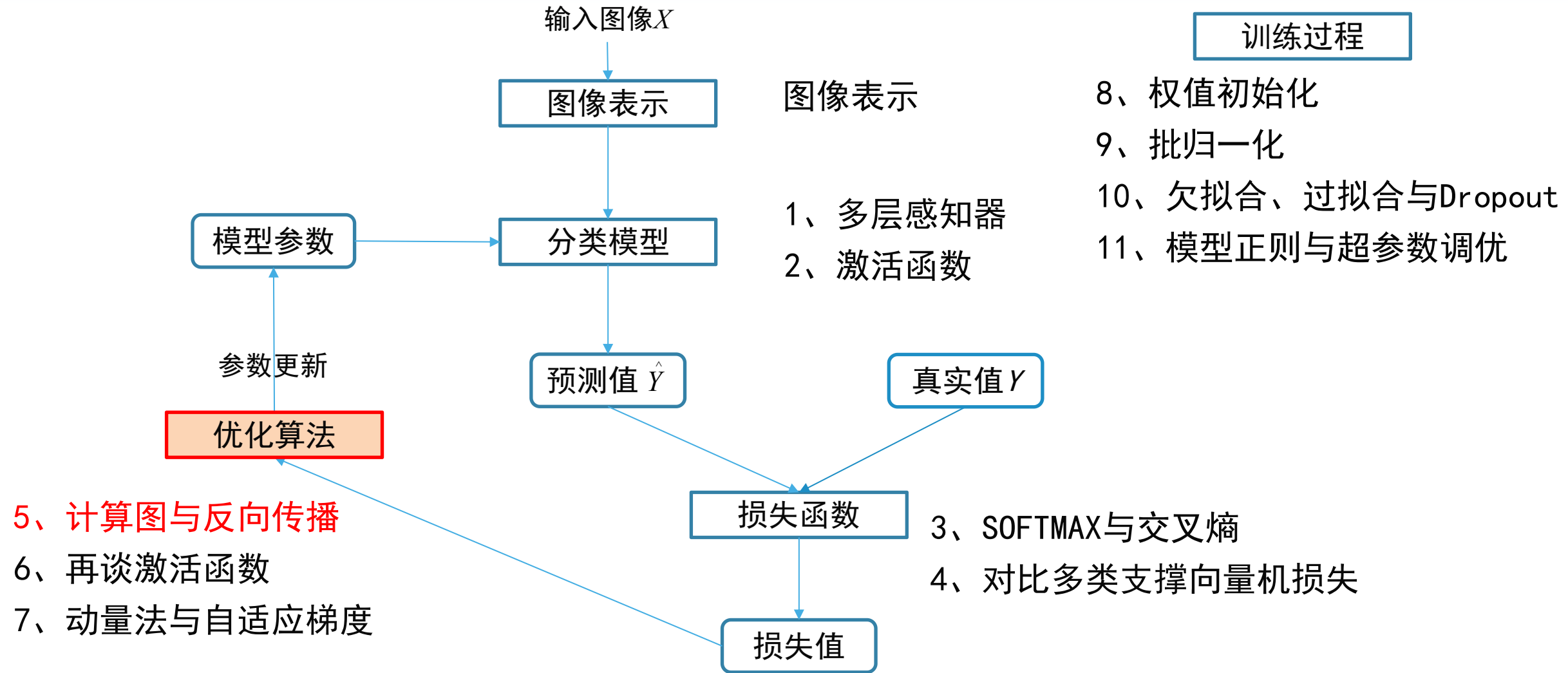
0

[0.35 0.33 0.32]

[0.333 0.333 0.334]



全连接神经网络设计与学习



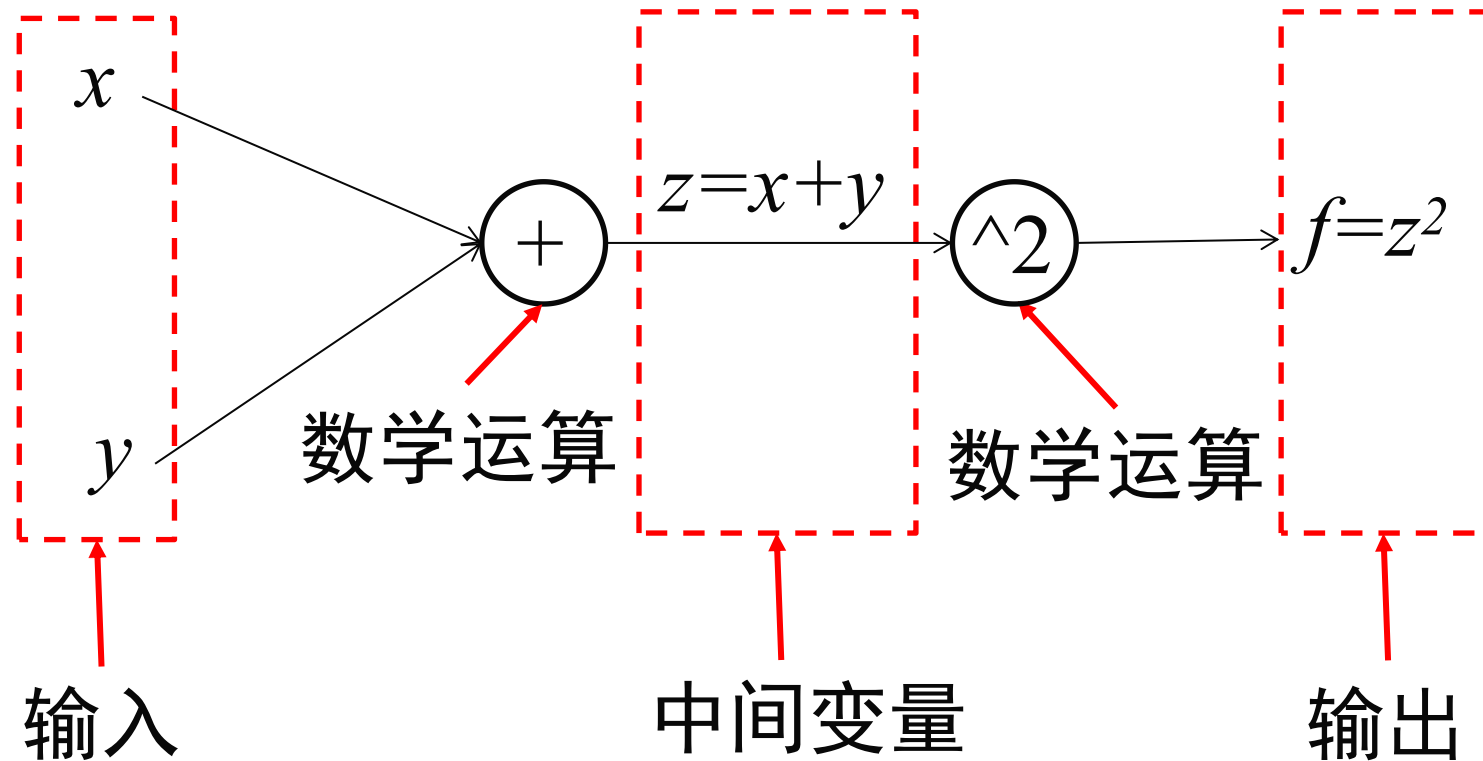
什么是计算图？

计算图是一种有向图，它用来表达输入、输出以及中间变量之间的计算关系，图中的每个节点对应着一种数学运算。



计算图与反向传播算法

函数 $f = (x+y)^2$ 的计算图：

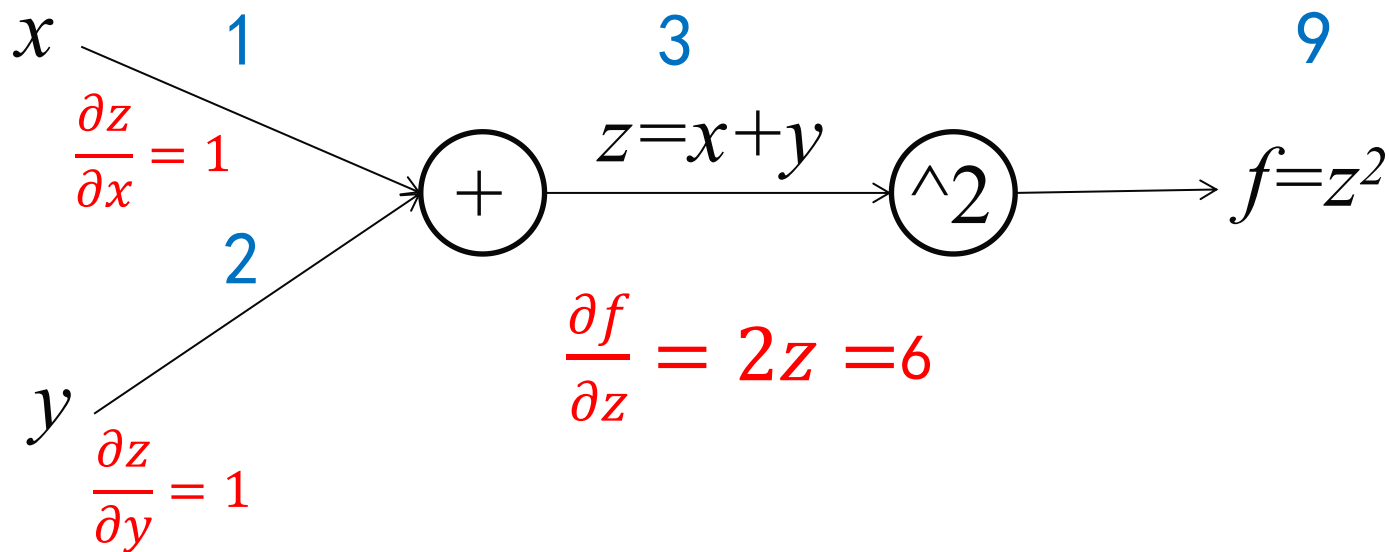


计算图的前向和反向计算示例

函数 $f=(x+y)^2$ 的计算图：

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial z} \frac{\partial z}{\partial x}$$

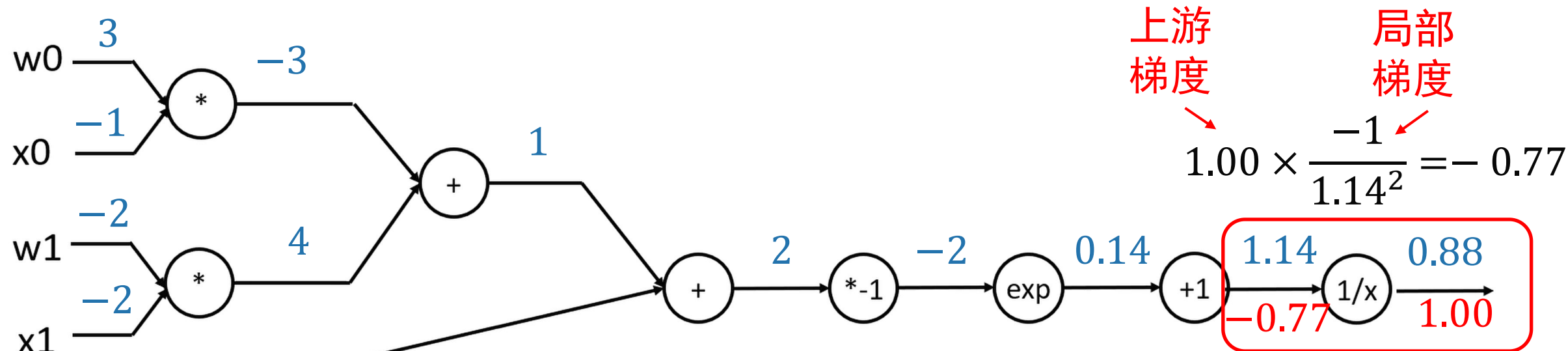
$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial z} \frac{\partial z}{\partial y}$$



计算图的反向传播示例2



$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{dy}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{dy}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{dy}{dx} = -1/x^2$$

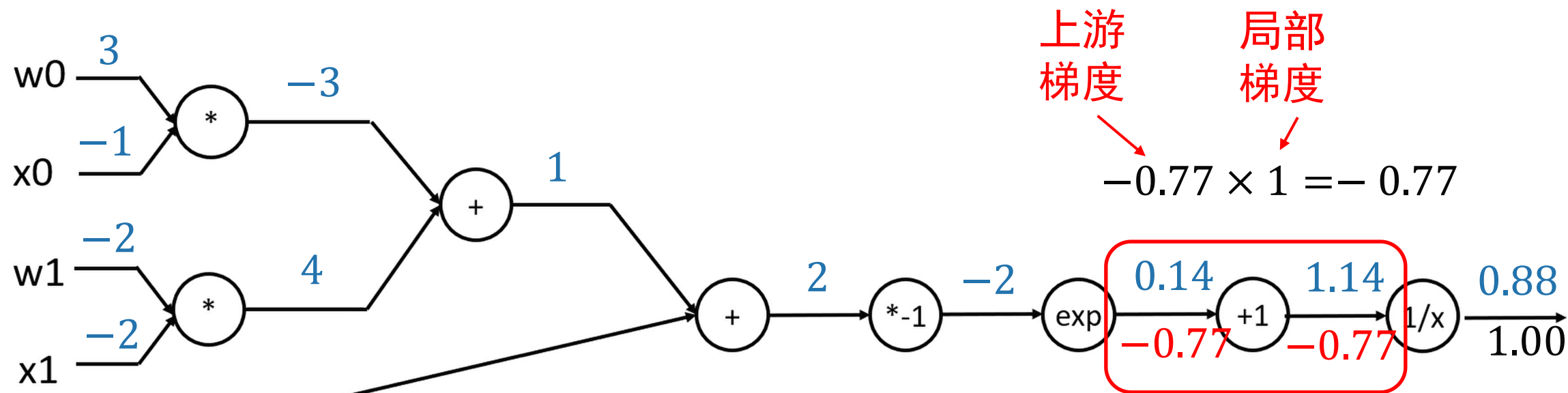
$$f_c(x) = c + x \rightarrow \frac{dy}{dx} = 1$$



计算图的反向传播示例2



$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{dy}{dx} = e^x$$

$$f(x) = \frac{1}{x} \rightarrow \frac{dy}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{dy}{dx} = a$$

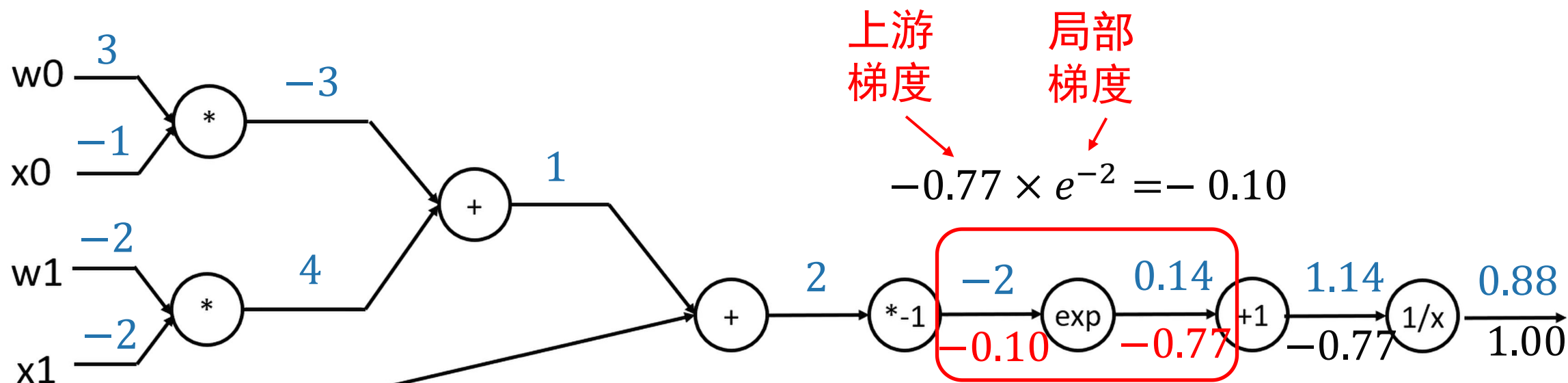
$$f_c(x) = c + x \rightarrow \frac{dy}{dx} = 1$$



计算图的反向传播示例2



$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{dy}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{dy}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{dy}{dx} = -1/x^2$$

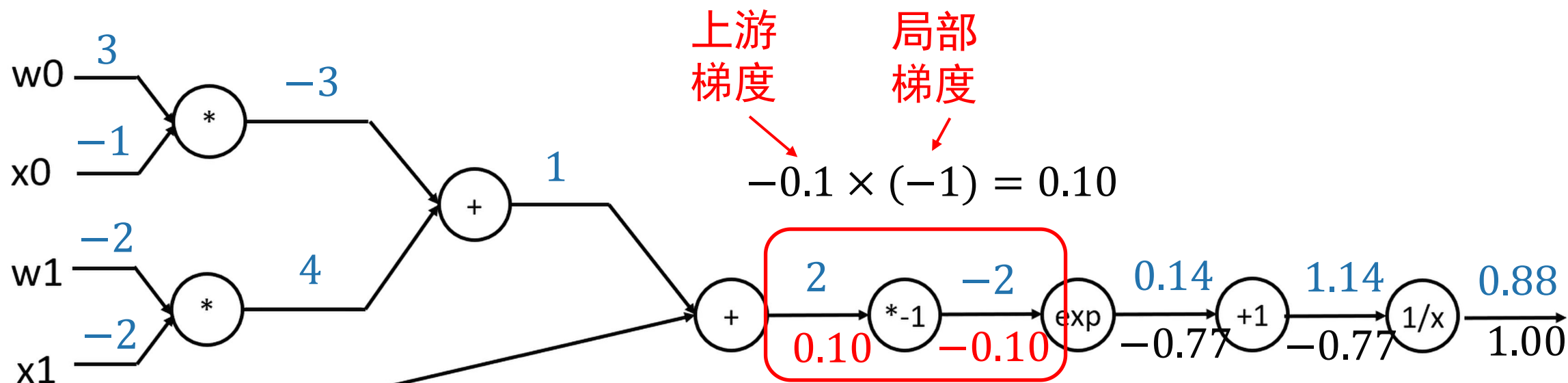
$$f_c(x) = c + x \rightarrow \frac{dy}{dx} = 1$$



计算图的反向传播示例2



$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{dy}{dx} = e^x$$

$$f(x) = \frac{1}{x} \rightarrow \frac{dy}{dx} = -1/x^2$$

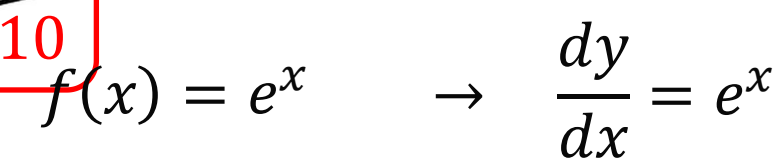
$$f_a(x) = ax \rightarrow \frac{dy}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{dy}{dx} = 1$$



上游梯度 局部梯度

$-0.1 \times 1 = 0.10$



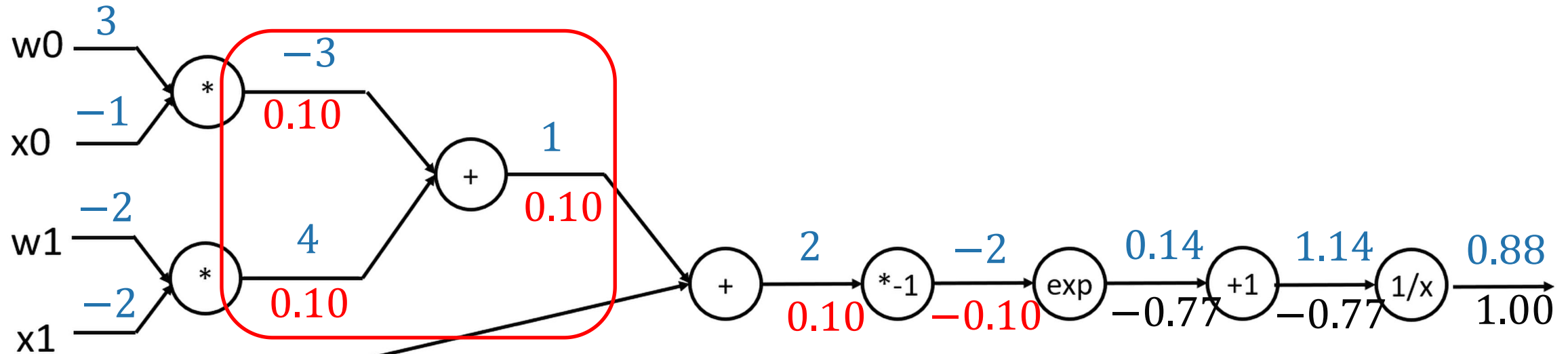
$$f_a(x) = ax \quad \rightarrow \quad \frac{dy}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{dy}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{dy}{dx} = 1$$

计算图的反向传播示例2

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{dy}{dx} = e^x$$

$$f(x) = \frac{1}{x} \rightarrow \frac{dy}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{dy}{dx} = a$$

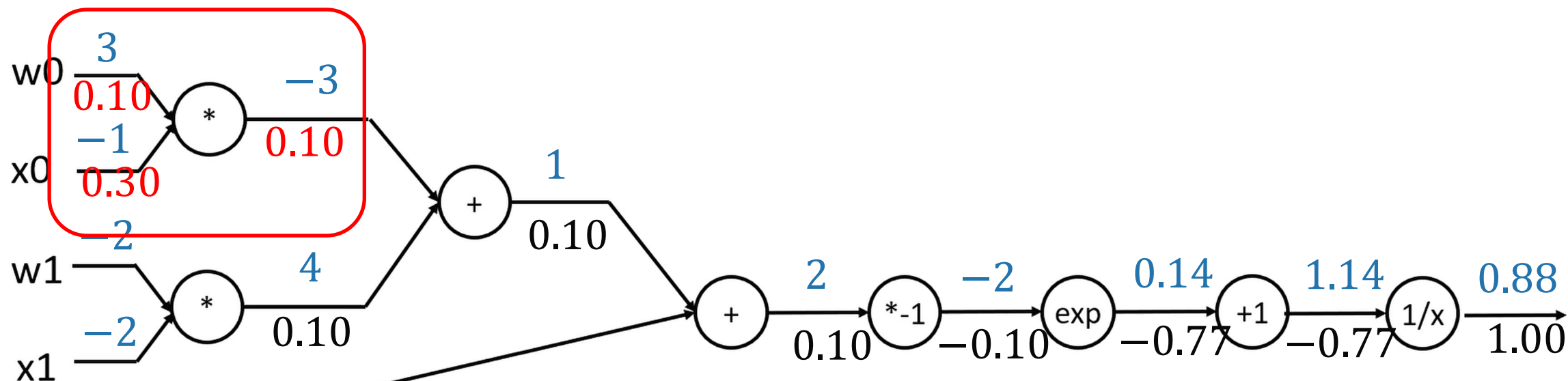
$$f_c(x) = c + x \rightarrow \frac{dy}{dx} = 1$$



计算图的反向传播示例2



$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$f(x) = e^x \rightarrow \frac{dy}{dx} = e^x$$

$$f(x) = \frac{1}{x} \rightarrow \frac{dy}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{dy}{dx} = a$$

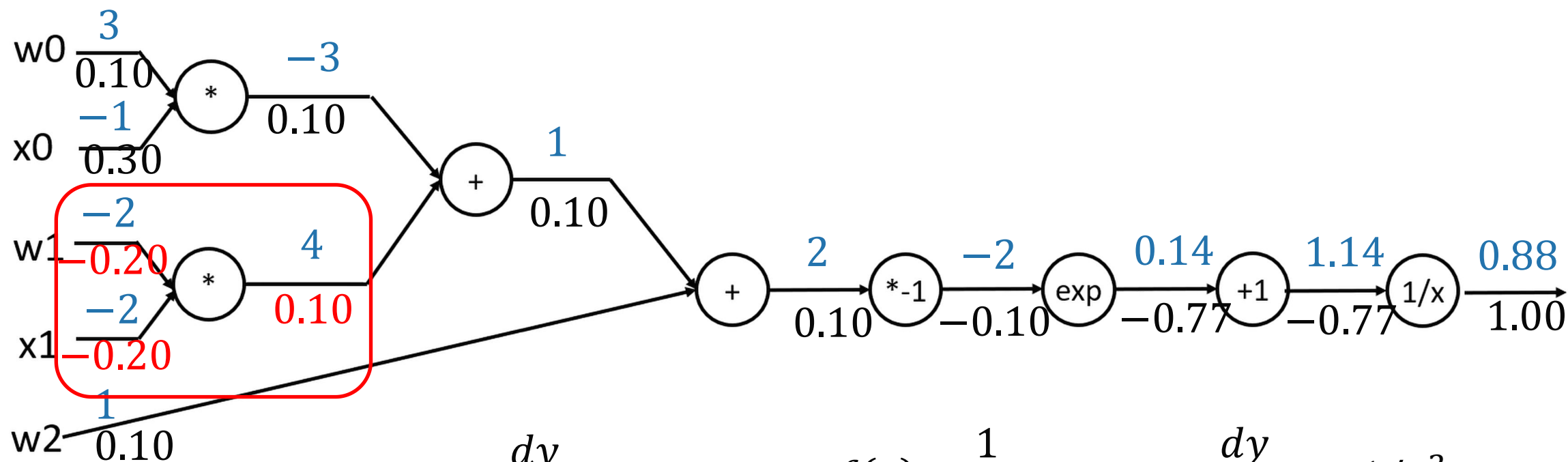
$$f_c(x) = c + x \rightarrow \frac{dy}{dx} = 1$$



计算图的反向传播示例2



$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$f(x) = e^x \rightarrow \frac{dy}{dx} = e^x$$

$$f(x) = \frac{1}{x} \rightarrow \frac{dy}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{dy}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{dy}{dx} = 1$$



计算图总结

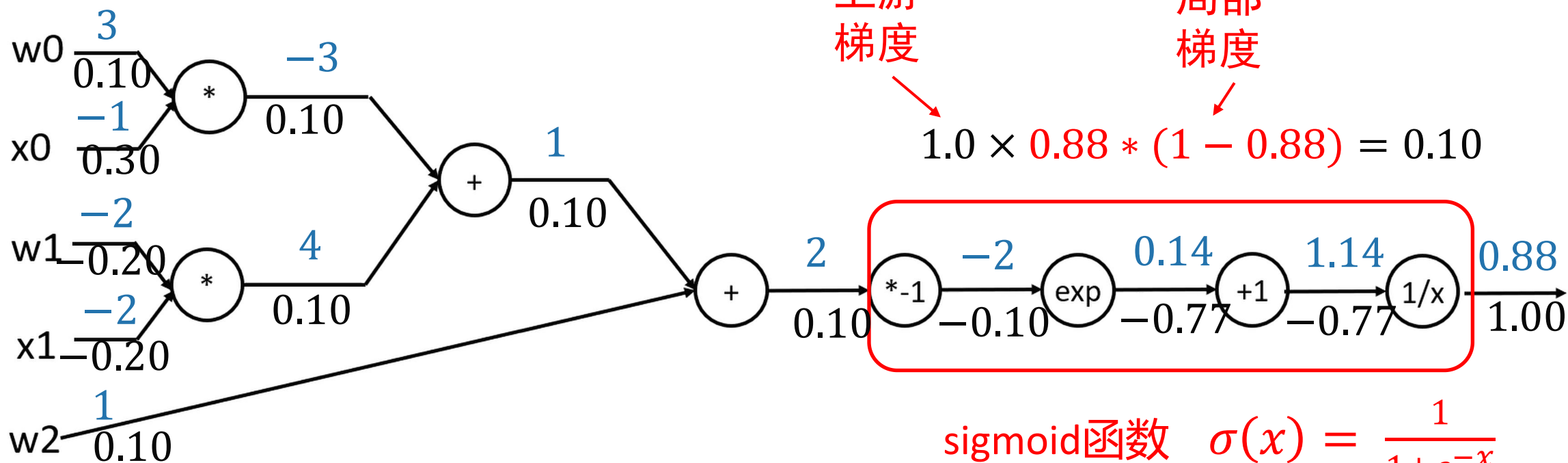
- ❖ 任意复杂的函数，都可以用计算图的形式表示
- ❖ 在整个计算图中，每个门单元都会得到一些输入，然后，进行下面两个计算：
 - a) 这个门的输出值
 - b) 其输出值关于输入值的局部梯度。
- ❖ 利用链式法则，门单元应该将回传的梯度乘以对其输入的局部梯度，从而得到整个网络的输出对该门单元的每个输入值的梯度。



计算机图的颗粒度



$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$

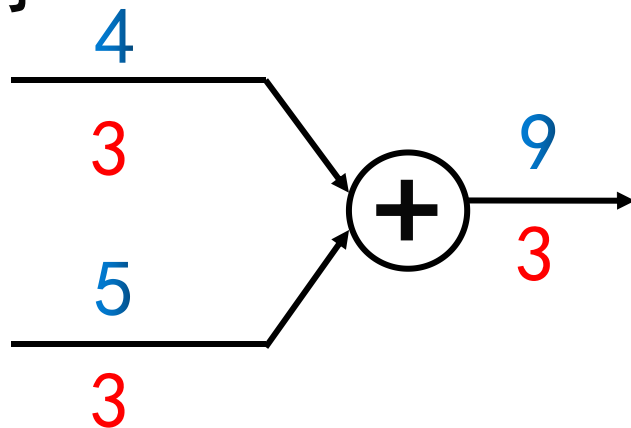


$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$$

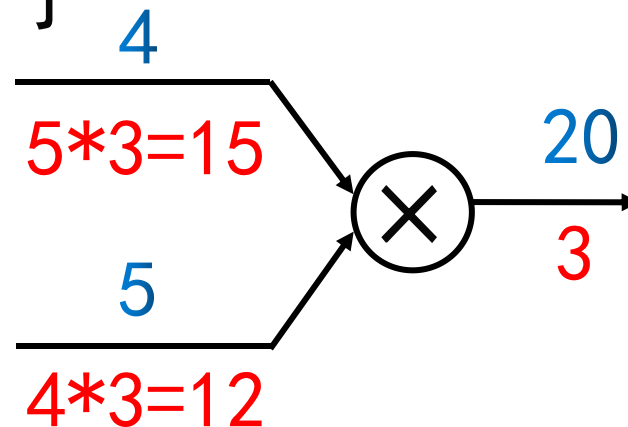


计算图中常见的门单元

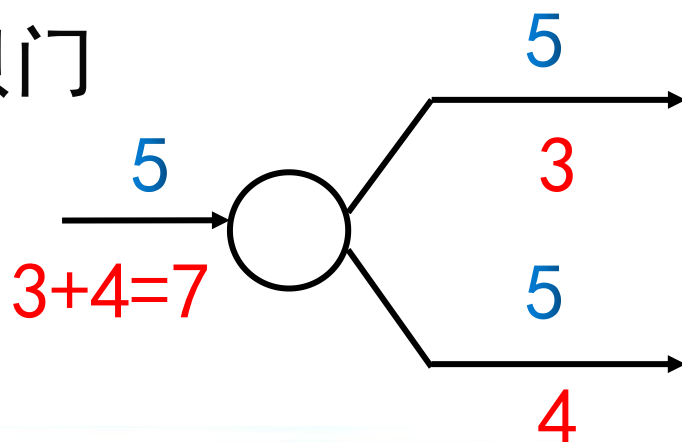
加法门



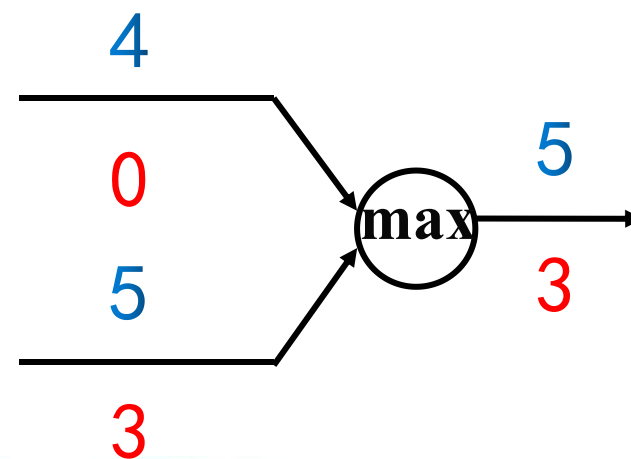
乘法门



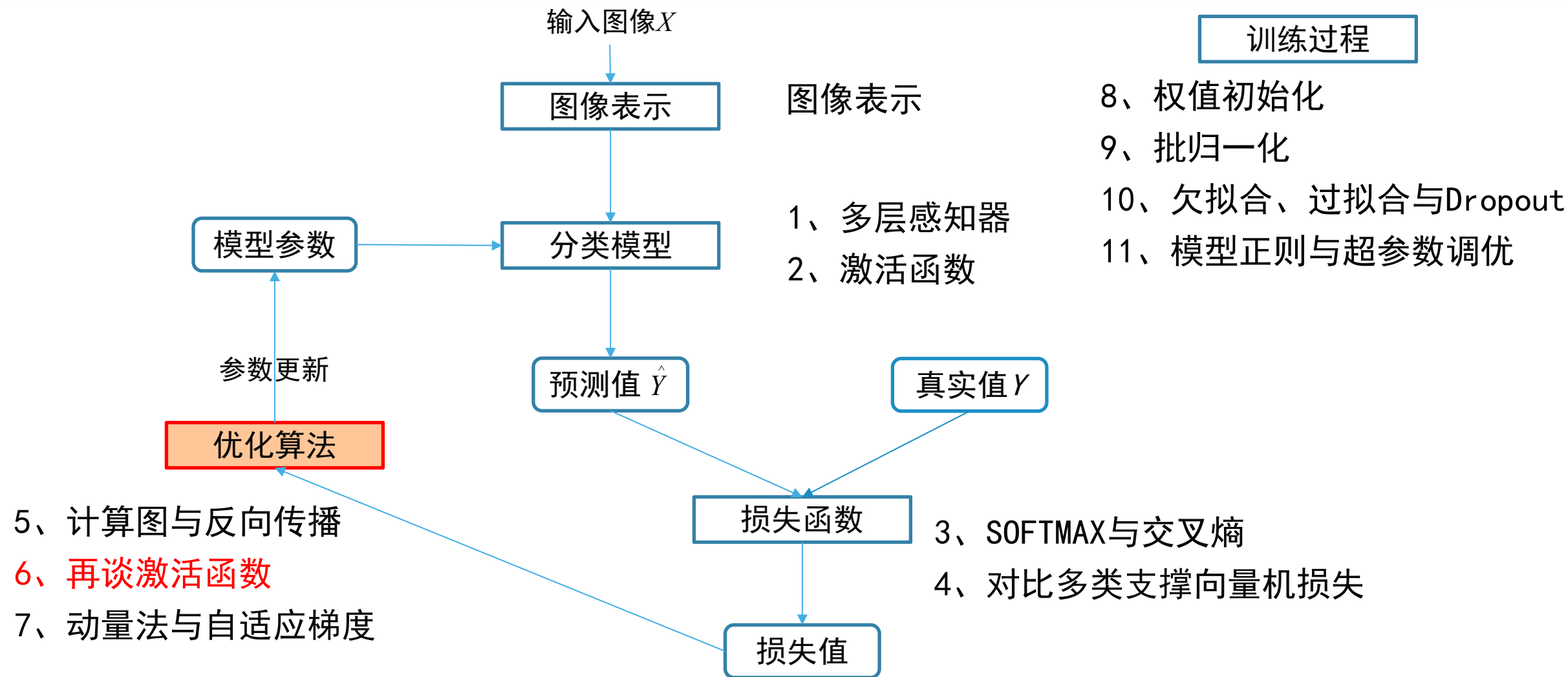
拷贝门



max门

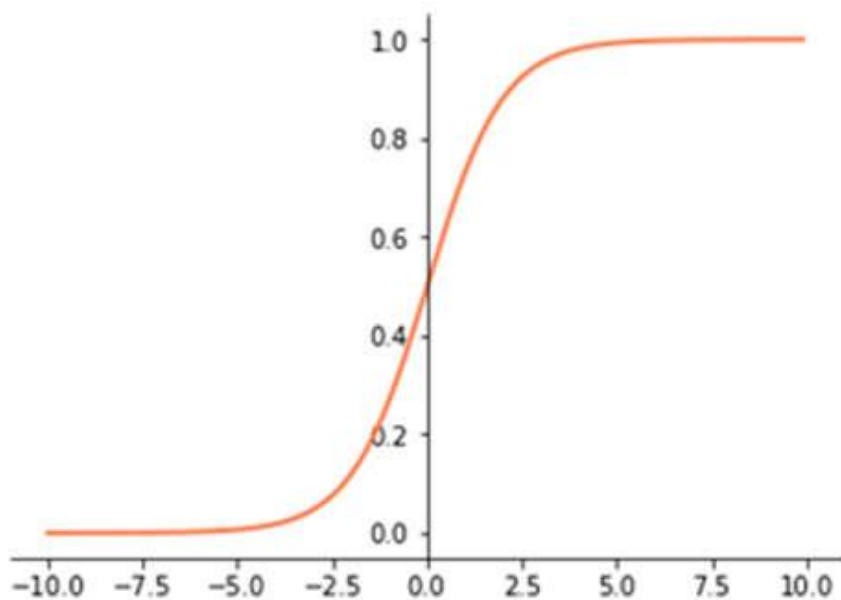


全连接神经网络设计与学习

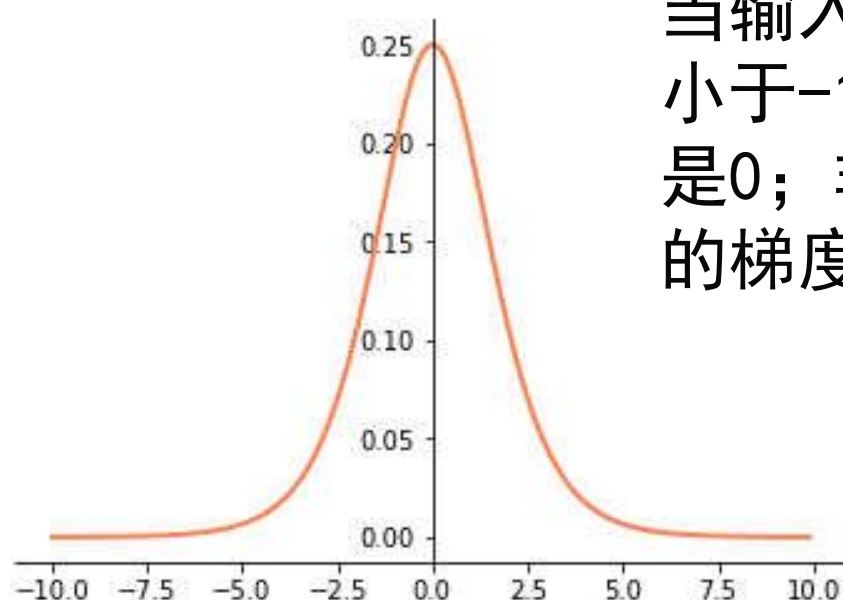


再看激活函数

Sigmoid激活函数: $\sigma(x) = 1/(1 + e^{-x})$



Sigmoid函数

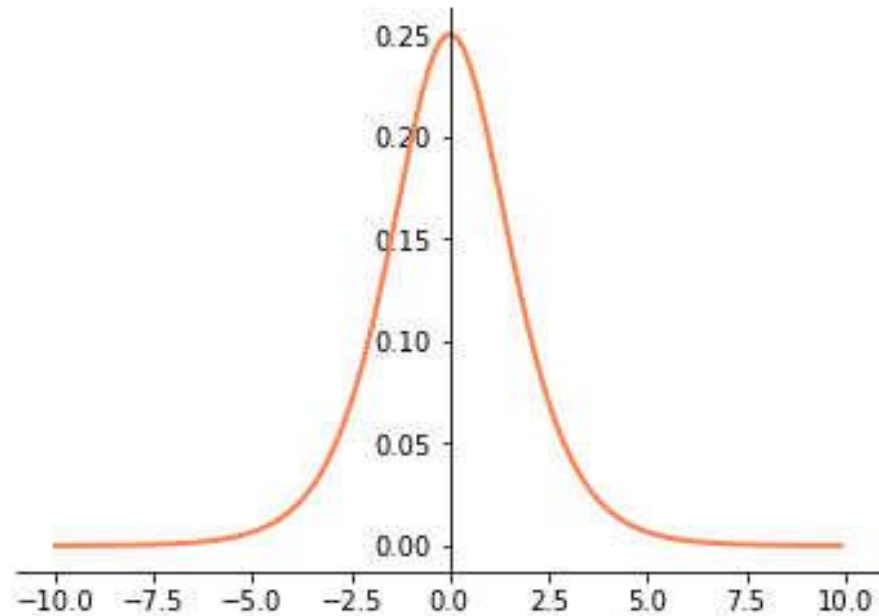


Sigmoid函数的导数函数

当输入值大于10或者小于-10时局部梯度都是0；非常不利于网络的梯度流传递的。

梯度消失

- ❖ 梯度消失是神经网络训练中非常致命的一个问题，其本质是由于链式法则的乘法特性导致的。

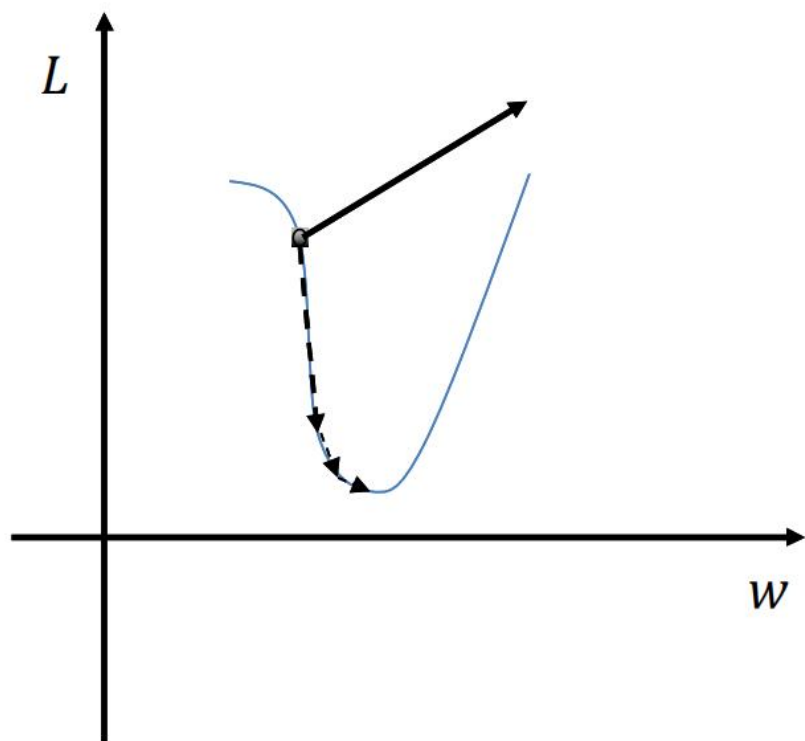


Sigmoid函数的导数函数

梯度爆炸



❖ 梯度爆炸也是由于链式法则的乘法特性导致的。

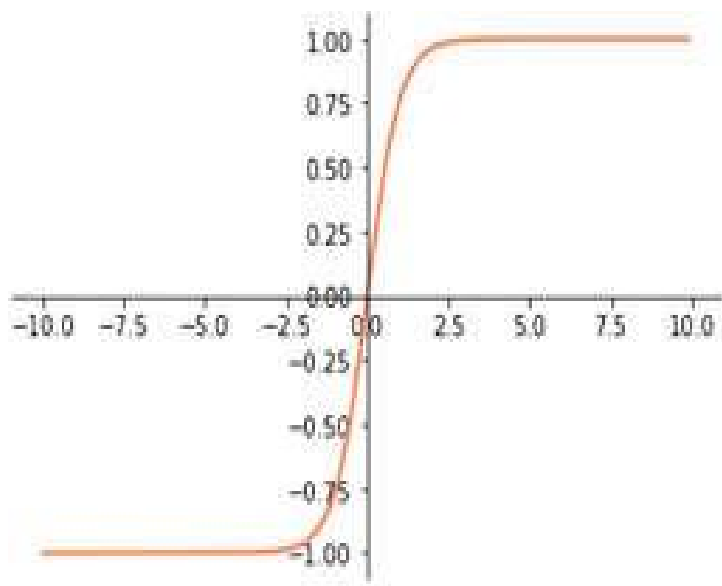


梯度爆炸：断崖处梯度乘以学习率后会是一个非常大得值，从而“飞”出了合理区域，最终导致算法不收敛；
解决方案：把沿梯度方向前进的步长限制在某个值内就可以避免“飞”出了，这个方法也称为梯度裁剪。

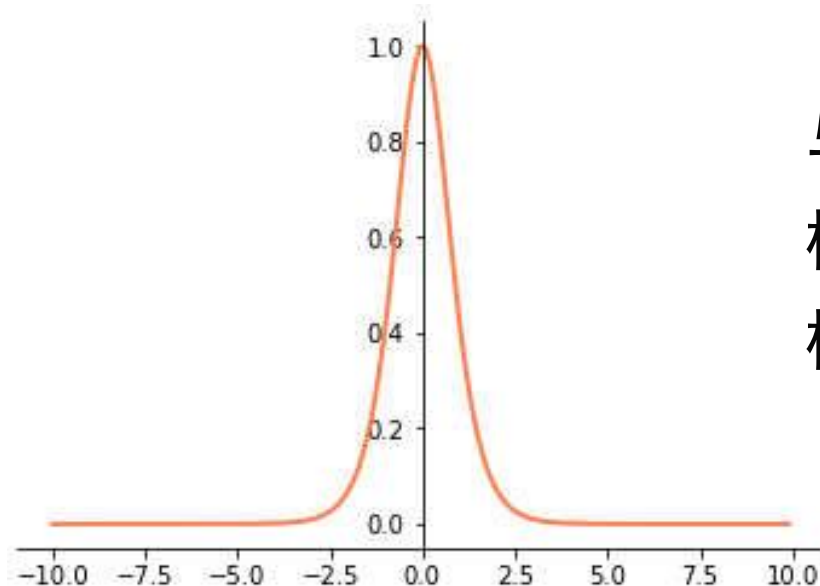


激活函数选择

双曲正切激活函数： $\tanh(x) = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$



tanh函数



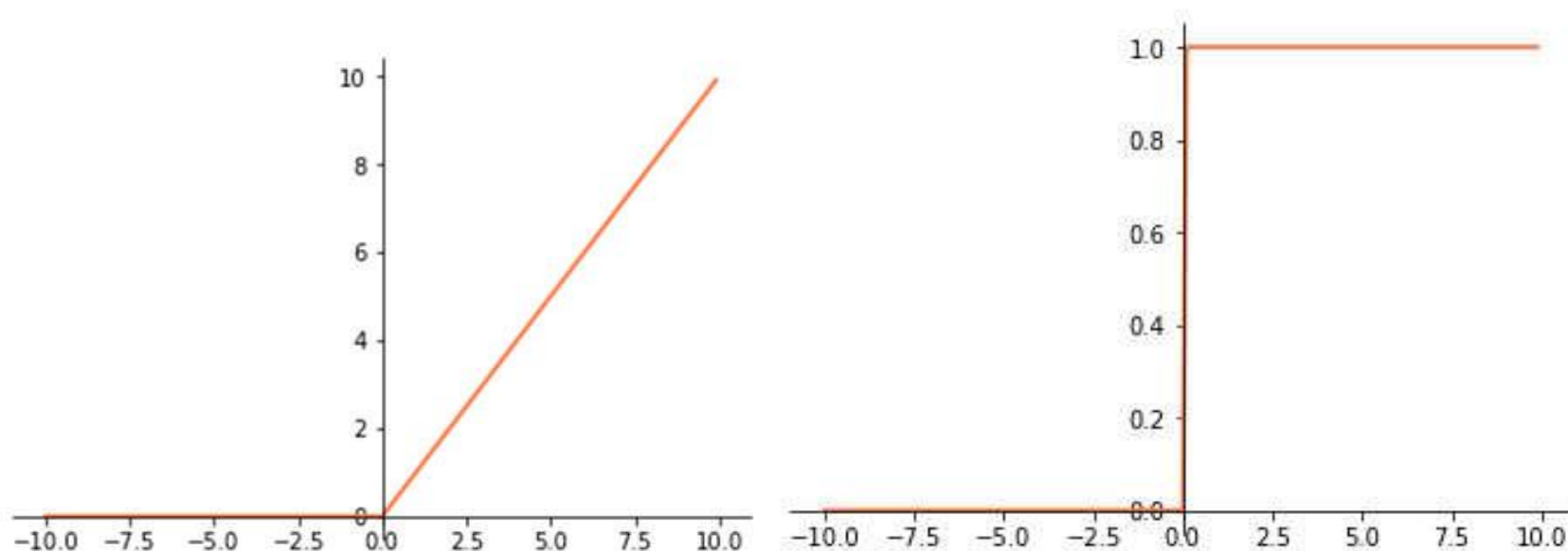
tanh函数的导数函数

与sigmoid类似，局部梯度特性不利于网络梯度流的反向传递。

激活函数选择



ReLU激活函数: $f(x) = \max(0, x)$



ReLU函数

ReLU函数的导数函数

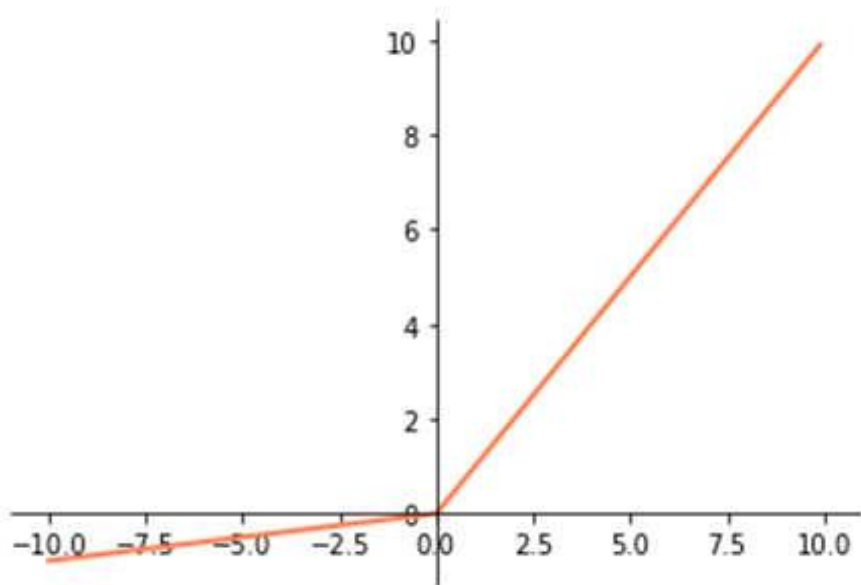
当输入大于0时，局部梯度永远不会为0，比较有利于梯度流的传递。



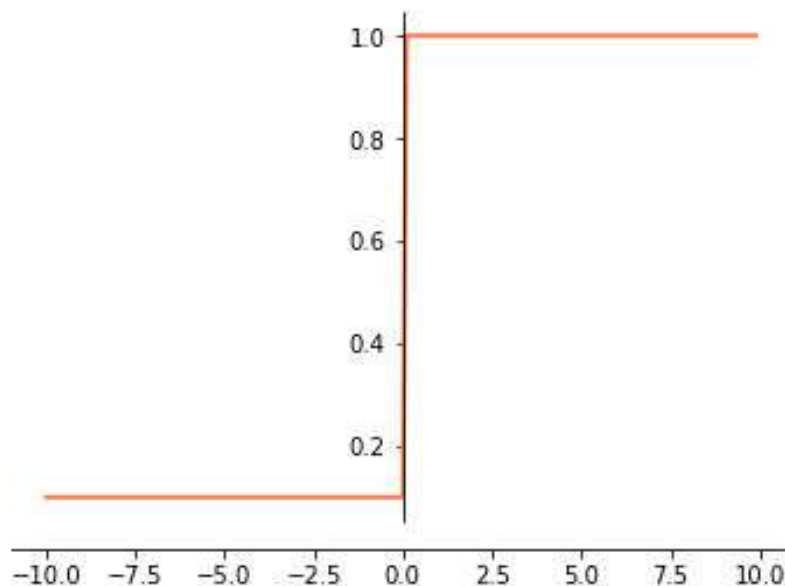
激活函数选择



Leaky ReLU激活函数: $f(x) = \max(0, x)$



Leaky ReLU函数



Leaky ReLU函数的导数函数

基本没有“死区”，也就是梯度永远不会为0。之所以说“基本”，是因为函数在0处没有导数。



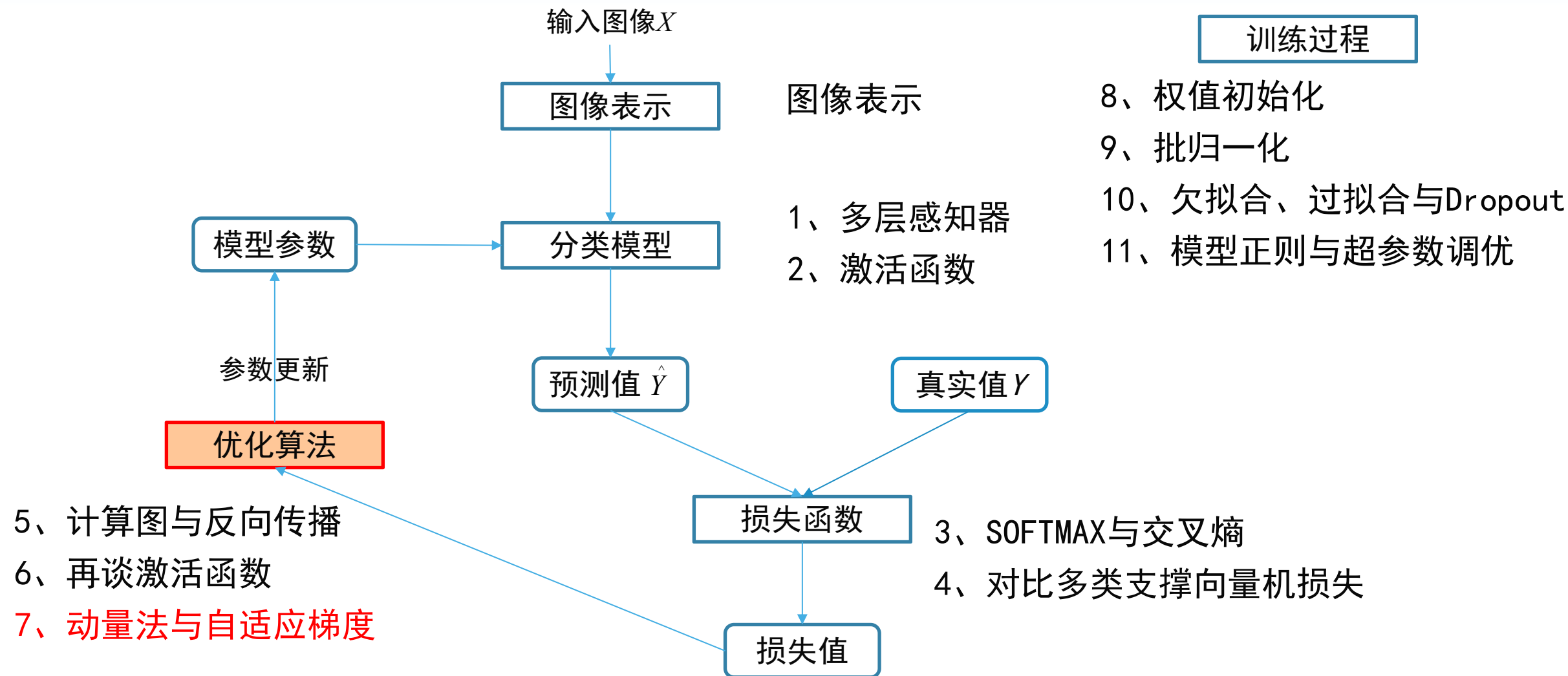


激活函数选择总结

尽量选择ReLU函数或者Leaky ReLU函数，相对于Sigmoid/tanh，ReLU函数或者Leaky ReLU函数会让梯度流更加顺畅，训练过程收敛得更快。



全连接神经网络设计与学习



梯度下降法存在的问题

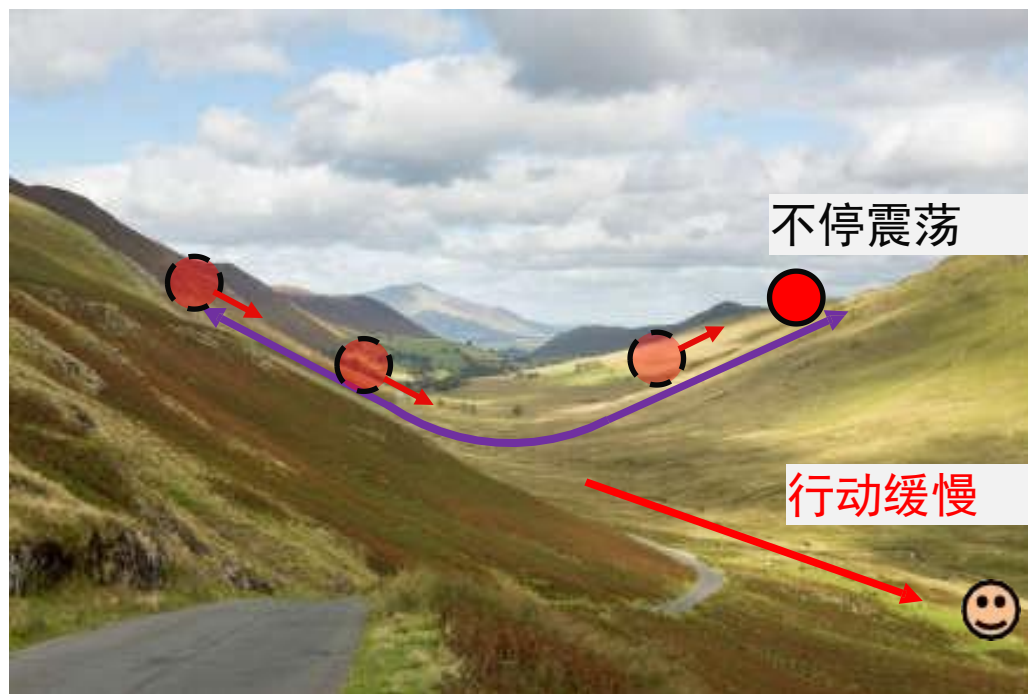
- ❖ 损失函数特性：一个方向上变化迅速而在另一个方向上变化缓慢。
- ❖ 优化目标：从起点处走到笑脸处



梯度下降法存在的问题

- ❖ 损失函数特性：一个方向上变化迅速而在另一个方向上变化缓慢。
- ❖ 优化目标：从起点处走到笑脸处
- ❖ 梯度下降算法存在的问题：山壁间震荡，往谷低方向的行进较慢。

仅增大步长并不能
加快算法收敛速度！



动量梯度下降法 (Momentum)

- ❖ 目标：改进梯度下降算法存在的问题，即减少震荡，
- ❖ 改进思想：利用累加历史梯度信息更新梯度

为什么有效？

累加过程中震荡方向相互抵消，平坦方向得到加强

使用动量的小批量梯度下降算法

Require: 学习率 ϵ , 动量系数 μ

Require: 初始参数 θ

初始化速度 $v = 0$

while 停止标准未满足 do:

1. 从训练集中采样 m (批量大小) 个样本

$\{x^{(1)}, \dots, x^{(m)}\}$, 对应的目标为 $\{y^{(1)}, \dots, y^{(m)}\}$

2. 计算梯度: $g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$

3. 速度更新: $v = \mu v + g$

4. 更新权值: $\theta \leftarrow \theta - \epsilon v$

end while

μ 取值范围 $[0, 1)$

$\mu=0$ 时等价于梯度下降算法

建议设置: 0.9

小批量梯度下降算法

Require: 学习率 ϵ

Require: 初始参数 θ

while 停止标准未满足 do:

1. 从训练集中采样 m (批量大小) 个样本

$\{x^{(1)}, \dots, x^{(m)}\}$, 对应的目标为 $\{y^{(1)}, \dots, y^{(m)}\}$

2. 计算梯度: $g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$

3. 更新权值: $\theta \leftarrow \theta - \epsilon g$

end while

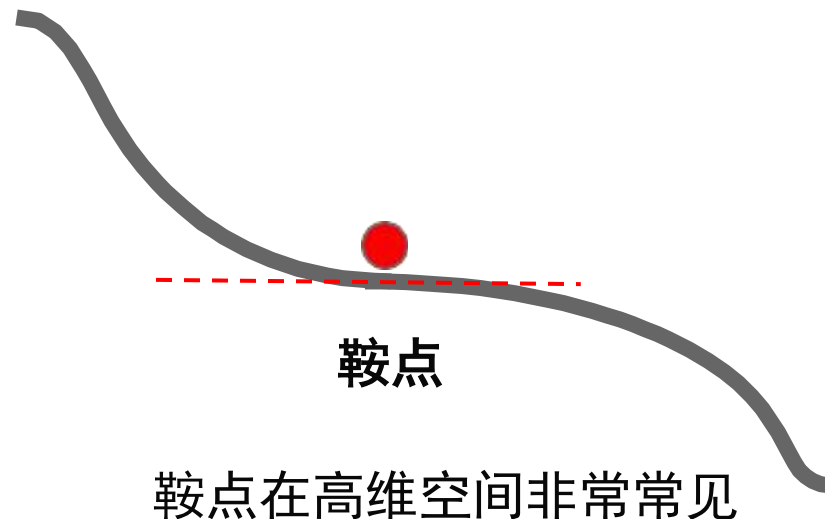
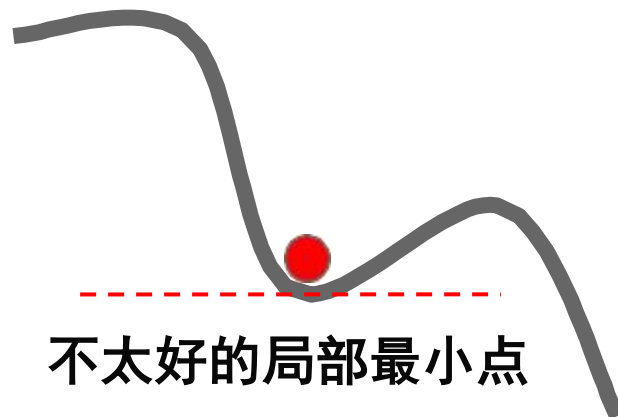


动量法还有什么效果？

现象：损失函数常具有不太好的局部最小值或鞍点（高维空间非常常见）

梯度下降算法存在的问题：局部最小处与鞍点处梯度为0，算法无法通过。

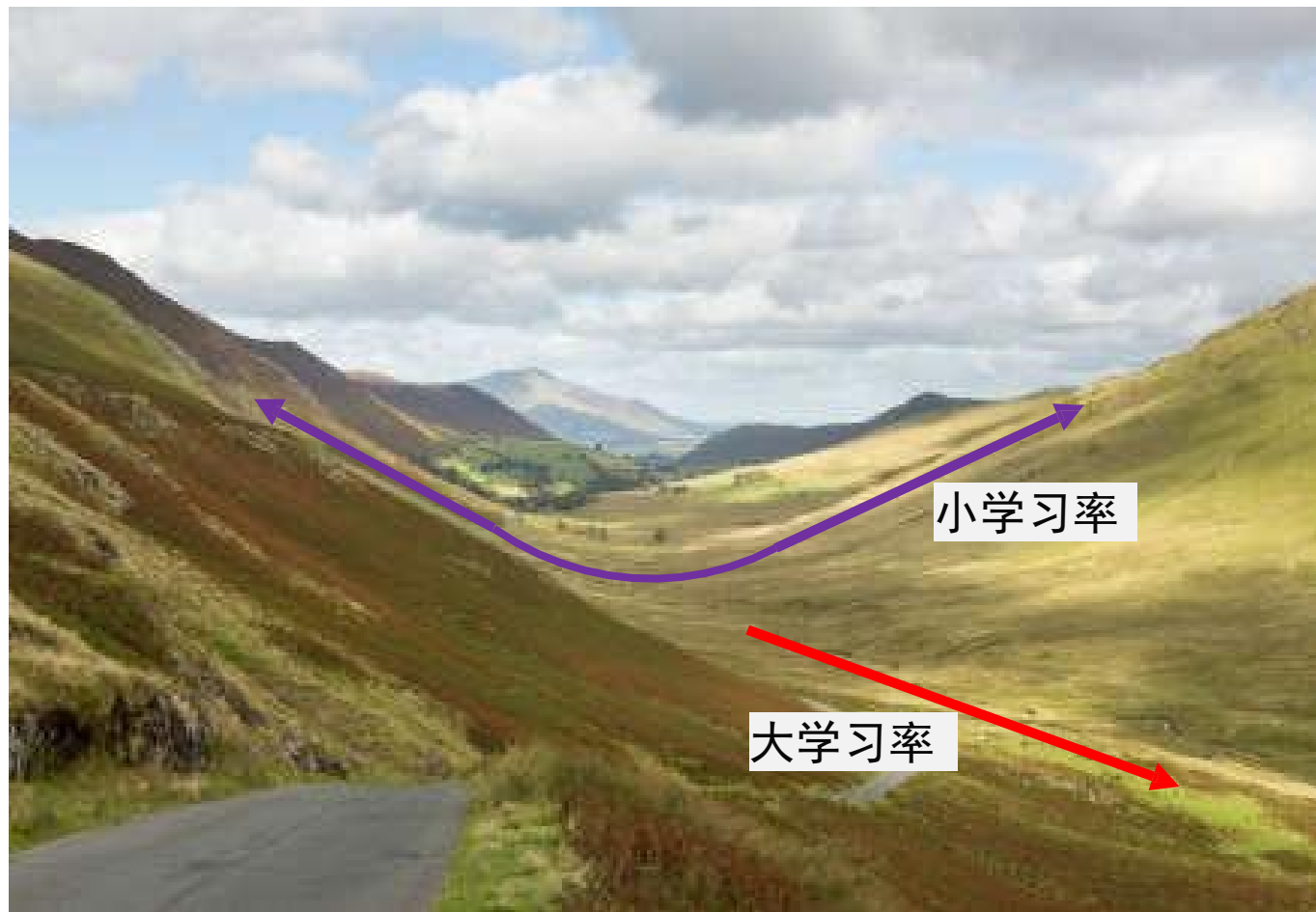
动量法的优势：由于动量的存在，算法可以冲出局部最小点以及鞍点，找到更优的解。



自适应梯度法



- 自适应梯度法通过减小震荡方向步长，增大平坦方向步长来减小震荡，加速通往谷底方向；
- 如何区分震荡方向与平坦方向？
回答：梯度幅度的平方较大的方向是震荡方向；梯度幅度的平方较小的方向是平坦方向。



AdaGrad梯度下降法

❖ AdaGrad方法是一种自适应梯度方法

AdaGrad算法

Require: 学习率 ϵ , 衰减速率 ρ

Require: 初始参数 θ

Require: 小常数 δ , 用于被小数除时的数值稳定 (通常设为 10^{-5})

初始化累积变量 $r = 0$

while 停止标准未满足 do:

1. 从训练集中采样 m (批量大小) 个样本 $\{x^{(1)}, \dots, x^{(m)}\}$,
对应的目标为 $\{y^{(1)}, \dots, y^{(m)}\}$

2. 计算梯度: $g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$

3. 累积平方梯度: $r \leftarrow r + g * g$

4. 更新权值: $w \leftarrow w - \frac{\epsilon}{\sqrt{r} + \delta} g$

end while

小批量梯度下降算法

Require: 学习率 ϵ

Require: 初始参数 θ

while 停止标准未满足 do:

1. 从训练集中采样 m (批量大小) 个样本 $\{x^{(1)}, \dots, x^{(m)}\}$, 对应的目标为 $\{y^{(1)}, \dots, y^{(m)}\}$

2. 计算梯度: $g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$

3. 更新权值: $\theta \leftarrow \theta - \epsilon g$

end while



RMSProp梯度下降法

❖ RMSProp方法是一种自适应梯度方法

RMSProp算法

Require: 学习率 ε , 衰减速率 ρ

Require: 初始参数 θ

Require: 小常数 δ , 用于被小数除时的数值稳定 (通常设为 10^{-5})

初始化累积变量 $r = 0$

while 停止标准未满足 do:

1. 从训练集中采样 m (批量大小) 个样本 $\{x^{(1)}, \dots, x^{(m)}\}$,
对应的目标为 $\{y^{(1)}, \dots, y^{(m)}\}$

2. 计算梯度: $g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta); y^{(i)})$

3. 累积平方梯度: $r \leftarrow \rho r + (1 - \rho) g * g$

4. 更新权值: $w \leftarrow w - \frac{\varepsilon}{\sqrt{r} + \delta} g$

end while

ρ 取值范围 $[0, 1)$

$\rho = 0$ 时仅考虑当前梯度的强度
建议设置: 0.999

小批量梯度下降算法

Require: 学习率 ε

Require: 初始参数 θ

while 停止标准未满足 do:

1. 从训练集中采样 m (批量大小) 个样本 $\{x^{(1)}, \dots, x^{(m)}\}$, 对应的目标为 $\{y^{(1)}, \dots, y^{(m)}\}$

2. 计算梯度: $g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$

3. 更新权值: $\theta \leftarrow \theta - \varepsilon g$

end while



Adam梯度下降法

Adam算法

Require: 学习率 ϵ , 衰减速率 ρ (建议值0.999), 动量系数 μ (建议值0.9)

Require: 初始参数 θ

Require: 小常数 δ , 用于被小数除时的数值稳定 (通常设为 10^{-5})

初始化累积变量 $r = 0, v = 0$

while 停止标准未满足 do:

1. 从训练集中采样 m (批量大小) 个样本 $\{x^{(1)}, \dots, x^{(m)}\}$, 对应的目标为 $\{y^{(1)}, \dots, y^{(m)}\}$

2. 计算梯度: $g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta); y^{(i)})$

3. 累积梯度: $v \leftarrow \mu v + (1 - \mu)g$

4. 累积平方梯度: $r \leftarrow \rho r + (1 - \rho)g * g$

5. 修正偏差: $\tilde{v} = \frac{v}{1 - \mu^t}$; $\tilde{r} = \frac{r}{1 - \rho^t}$

6. 更新权值: $\theta \leftarrow \theta - \frac{\epsilon}{\sqrt{\tilde{r} + \delta}} \tilde{v}$

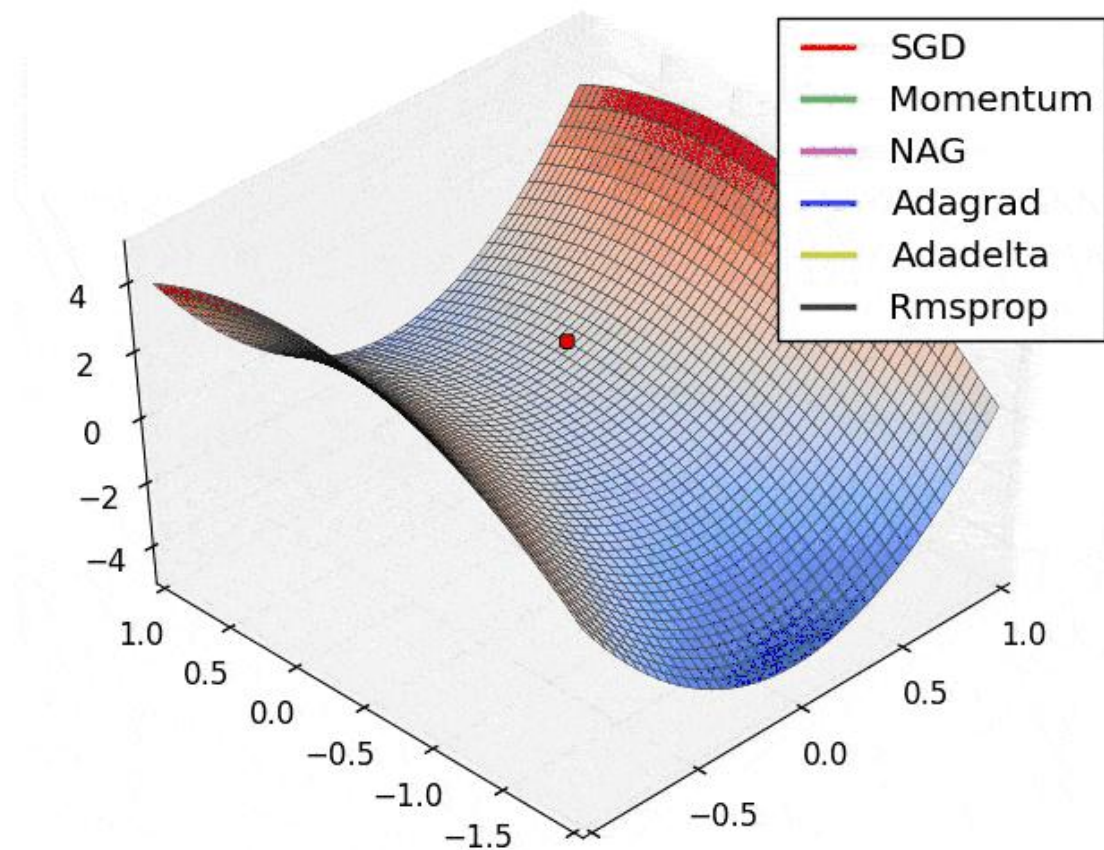
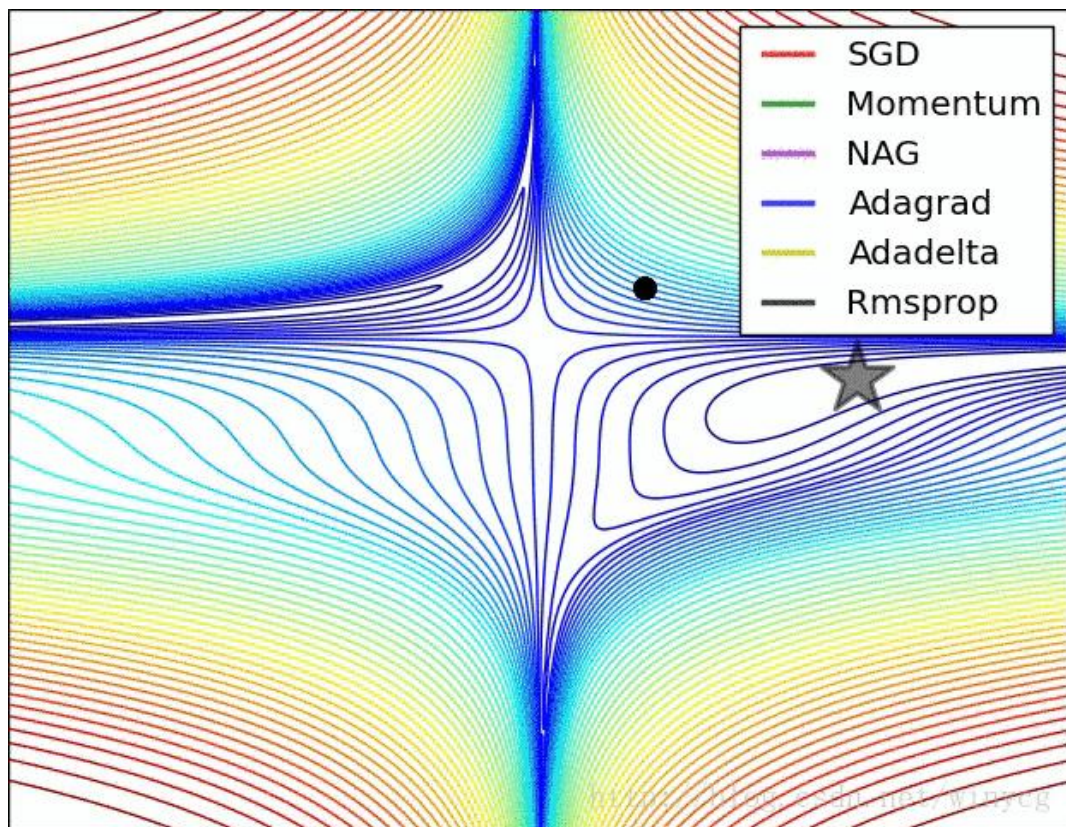
end while

❖ 同时使用动量与自适应梯度思想

❖ 修正偏差步骤可以极大缓解算法初期的冷启动问题



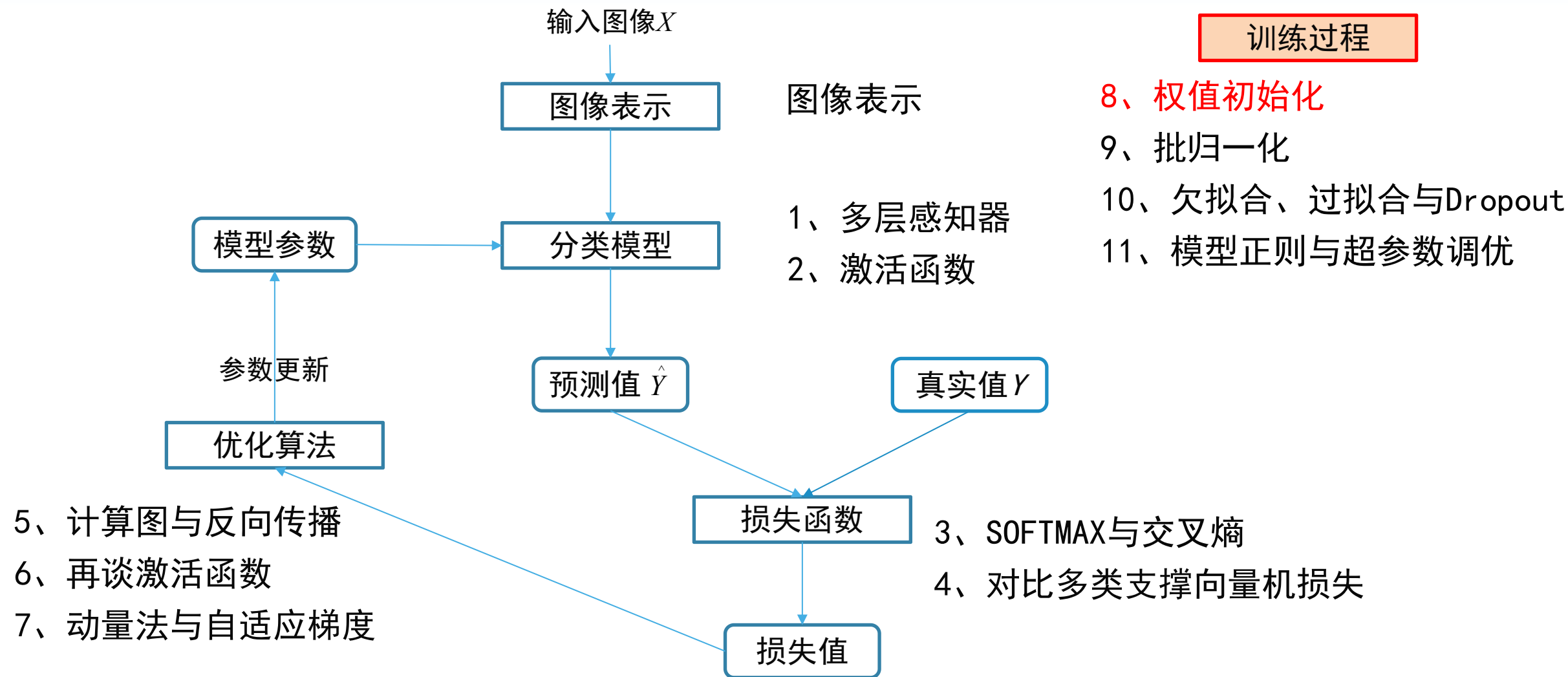
总结



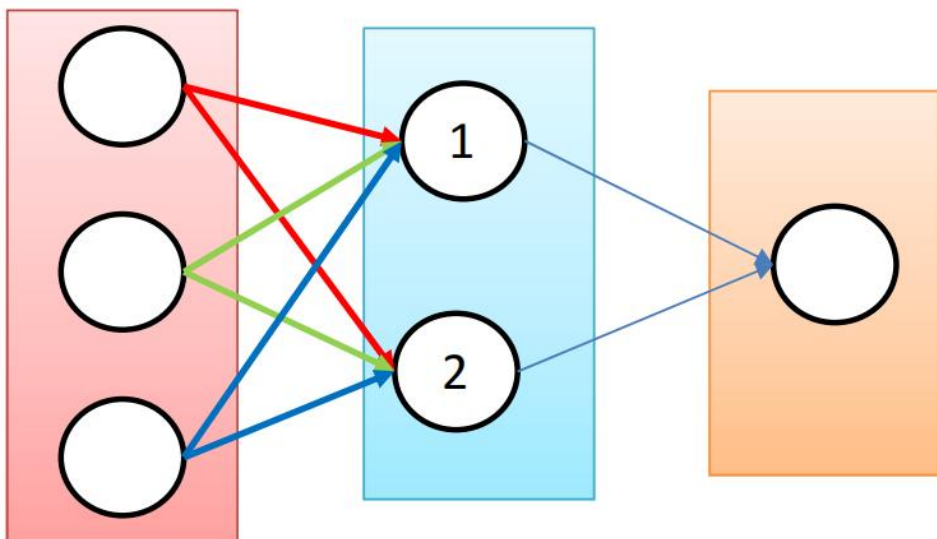
SGD各优化方法在损失曲面
鞍点处的表现



全连接神经网络设计与学习



权值初始化的问题：全零初始化



建议：采用随机初始化，
避免全零初始化！

全零初始化：网络中不同的神经元有相同的输出，进行同样的参数更新；
因此，这些神经元学到的参数都一样，等价于一个神经元

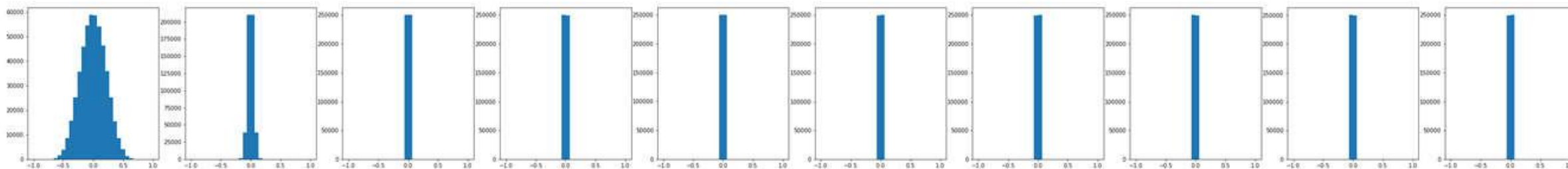
随机权值初始化

网络结构：10个隐层，1个输出层，每个隐层包含500个神经元，使用的双曲正切激活函数。

随机初始化：

权值采样自 $N(0, 0.01)$ 的高斯分布

```
input layer had mean 0.000097 and std 0.999691
hidden layer 1 mean -0.000134 and std 0.214021
hidden layer 2 mean 0.000025 and std 0.047655
hidden layer 3 mean -0.000030 and std 0.010683
hidden layer 4 mean -0.000002 and std 0.002390
hidden layer 5 mean -0.000000 and std 0.000532
hidden layer 6 mean -0.000000 and std 0.000119
hidden layer 7 mean -0.000000 and std 0.000027
hidden layer 8 mean 0.000000 and std 0.000006
hidden layer 9 mean 0.000000 and std 0.000001
hidden layer 10 mean 0.000000 and std 0.000000
```



除了前两层，后续所有层的激活值为0；此时，输入信息传递不到输出层；
最终，网络得不到训练

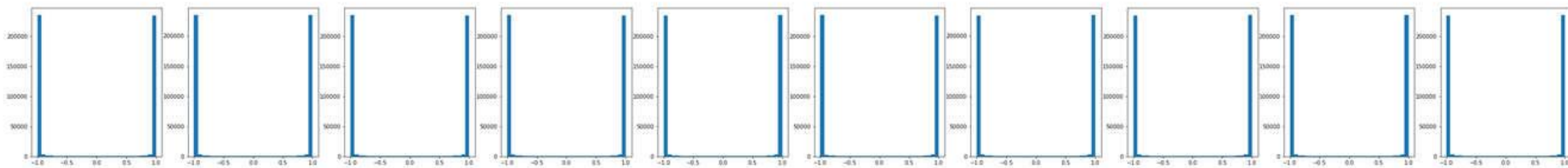
随机权值初始化

网络结构：10个隐层，1个输出层，每个隐层包含500个神经元，使用的双曲正切激活函数。

随机初始化：

权值采样自 $N(0, 1)$ 的高斯分布

```
input layer had mean 0.000240 and std 1.000121
hidden layer 1 mean -0.003320 and std 0.982045
hidden layer 2 mean 0.002391 and std 0.981602
hidden layer 3 mean 0.000433 and std 0.981700
hidden layer 4 mean 0.000040 and std 0.981740
hidden layer 5 mean 0.000358 and std 0.981791
hidden layer 6 mean -0.000324 and std 0.981612
hidden layer 7 mean 0.002807 and std 0.981484
hidden layer 8 mean -0.001022 and std 0.981578
hidden layer 9 mean 0.000497 and std 0.981684
hidden layer 10 mean -0.001396 and std 0.981915
```



几乎所有的神经元都饱和了（不是-1就是1）；此时，神经元局部梯度都是零，网络没有反向梯度流；最终，所有的参数得不到更新

随机权值初始化

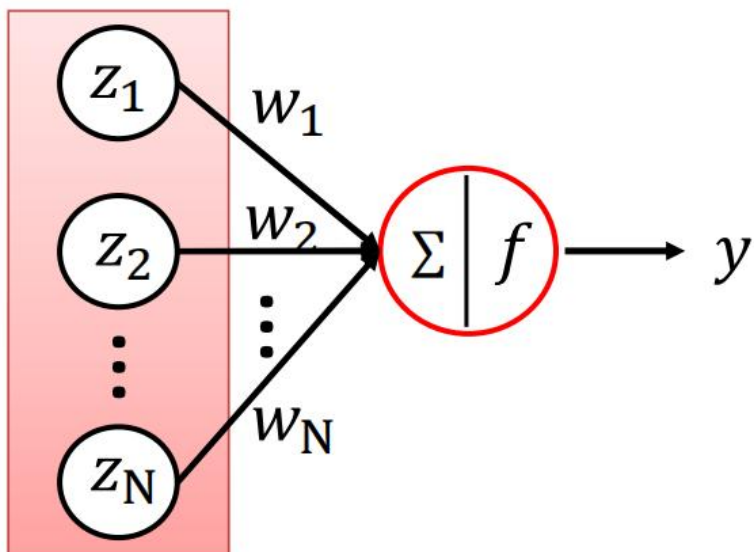
- ❖ **实验结论：**初始化时让权值不相等，并不能保证网络能够正常的被训练。
- ❖ **有效的初始化方法：**使网络各层的激活值和局部梯度的方差在传播过程中尽量保持一致；以保持网络中正向和反向数据流动



Xavier初始化

- ❖ 一个神经元，其输入为 $z_1, z_2, \dots, z_N$ ，这 N 个输入是独立同分布的；其权值为 $w_1, w_2, \dots, w_N$ ，它们也是独立同分布的，且 w 与 z 是独立的；激活函数为 f ；其最终输出 y 的表达式：

$$y = f(w_1 * z_1 + \dots + w_n * z_n)$$

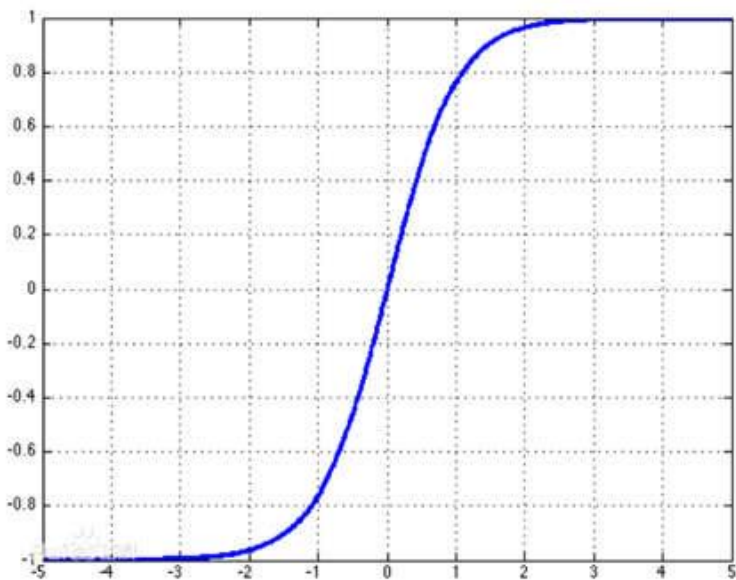


目标：使网络各层的激活值和局部梯度的方差在传播过程中尽量保持一致，即寻找 w 的分布使得输出 y 与输入 e 的方差一致

Xavier初始化

- ❖ 一个神经元，其输入为 $z_1, z_2, \dots, z_N$ ，这 N 个输入是独立同分布的；其权值为 $w_1, w_2, \dots, w_N$ ，它们也是独立同分布的，且 w 与 z 是独立的；激活函数为 f ；其最终输出 y 的表达式：

$$y = f(w_1 * z_1 + \dots + w_n * z_n)$$



假设 f 为双曲正切函数， $w_1, w_2, \dots, w_N$ 独立同分布， $z_1, \dots, z_N$ 独立同分布，随机变量 w 与 z 独立，且均值都为0，则有：

$$\begin{aligned} \text{Var}(y) &= \text{Var}\left(\sum_{i=1}^n w_i z_i\right) = \sum_{i=1}^n \text{Var}(w_i z_i) \\ &= \sum_i^n [E(w_i)]^2 \text{Var}(z_i) + [E(z_i)]^2 \text{Var}(w_i) + \text{Var}(w_i) \text{Var}(z_i) \\ &= \sum_i^n \text{Var}(w_i) \text{Var}(z_i) \\ &= n \text{Var}(w_i) \text{Var}(z_i) \end{aligned}$$

$\text{var}(w) = 1/N$ 时， y 的方差与 z 的方差一致。



Xavier初始化

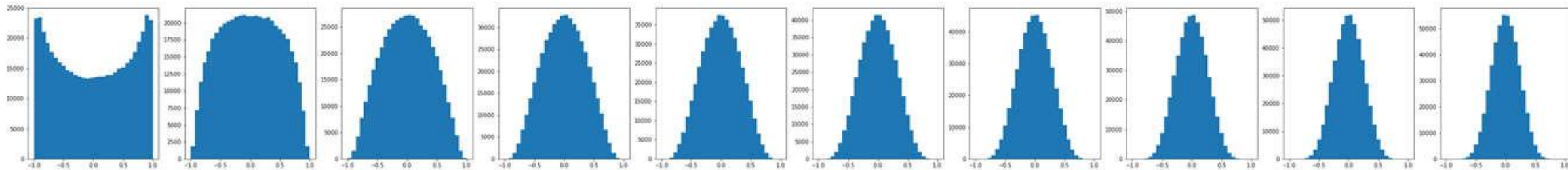
网络结构：10个隐层，1个输出层，每个隐层包含500个神经元，使用的双曲正切激活函数。

随机初始化：

权值采样自 $\mathcal{N}(0, 1/N)$ 的高斯分布, N 为输入神经元个数

达到目的：

每层神经元激活值的方差基本相同！



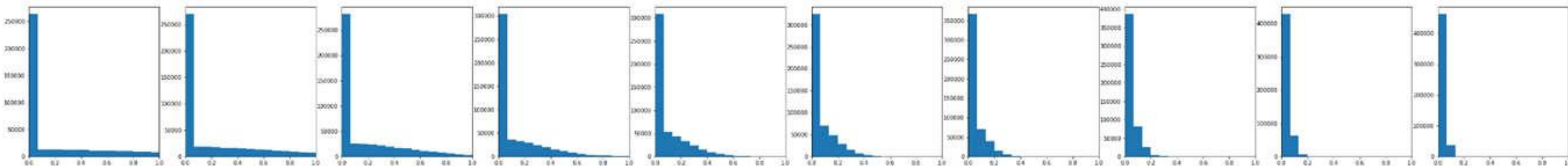
Xavier初始化

网络结构：10个隐层，1个输出层，每个隐层包含500个神经元，使用的ReLU激活函数。

随机初始化：

权值采样自 $\mathcal{N}(0, 1/N)$ 的高斯分布， N 为输入神经元个数

Xavier初始化方法不太适合ReLU
激活函数！



HE初始化 (MSRA)

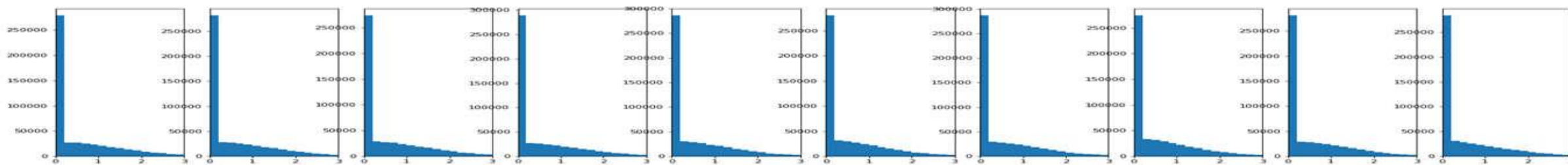
网络结构：10个隐层，1个输出层，每个隐层包含500个神经元，使用的ReLU激活函数。

随机初始化：

权值采样自 $\mathcal{N}(0, 2/N)$ 的高斯分布, N 为输入神经元个数

修改此处

ReLU激活函数初始化方法



权值初始化小结

- ❖ 好的初始化方法可以防止前向传播过程中的信息消失，也可以解决反向传递过程中的梯度消失。
- ❖ 激活函数选择双曲正切或者Sigmoid时，建议使用Xavier初始化方法；
- ❖ 激活函数选择ReLU或Leaky ReLU时，推荐使用He初始化方法。

全连接神经网络设计与学习



输入图像 X

图像表示

图像表示

分类模型

- 1、多层感知器
- 2、激活函数

预测值 $\hat{Y}$

真实值 Y

损失函数

- 3、SOFTMAX与交叉熵
- 4、对比多类支撑向量机损失

损失值

模型参数

参数更新

优化算法

- 5、计算图与反向传播
- 6、再谈激活函数
- 7、动量法与自适应梯度

训练过程

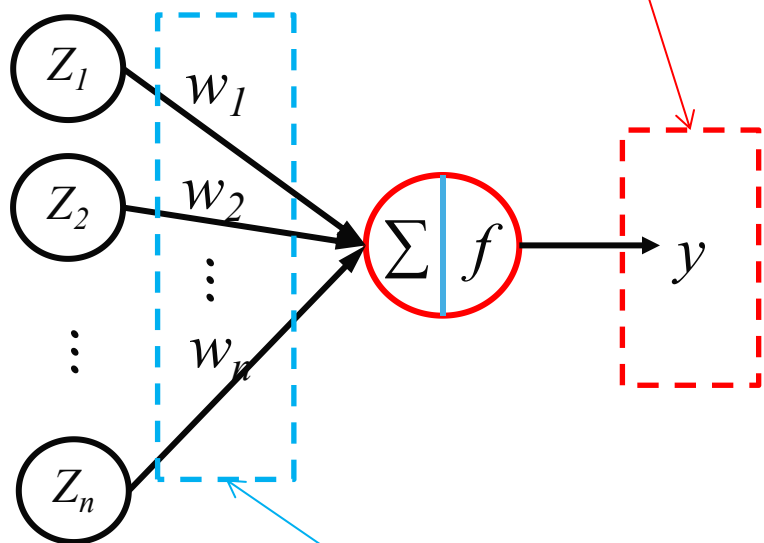
- 8、权值初始化
- 9、批归一化
- 10、欠拟合、过拟合与Dropout
- 11、模型正则与超参数调优



批归一化



批归一化方法
考虑的对象



权值初始化
考虑的对象

方法：调整权值分布使得输出与输入具有相同的分布。

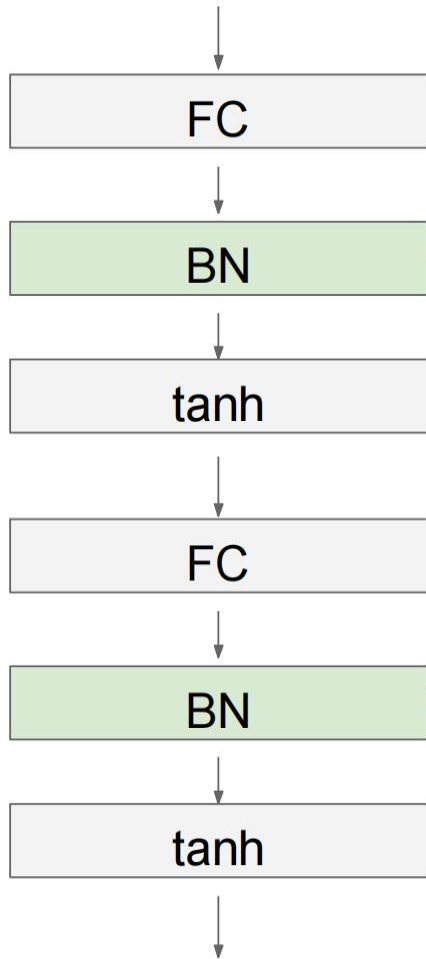
方法：直接对神经元的输出进行批归一化

- ❖ 小批量梯度下降算法回顾：每次迭代时会读入一批数据，比如32个样本；经过当前神经元后会有32个输出值 $y_1, \dots, y_{32}$ 。
- ❖ 批归一化操作：对这32个输出进行减均值除方差操作；可保证当前神经元的输出值的分布符合0 均值1方差。

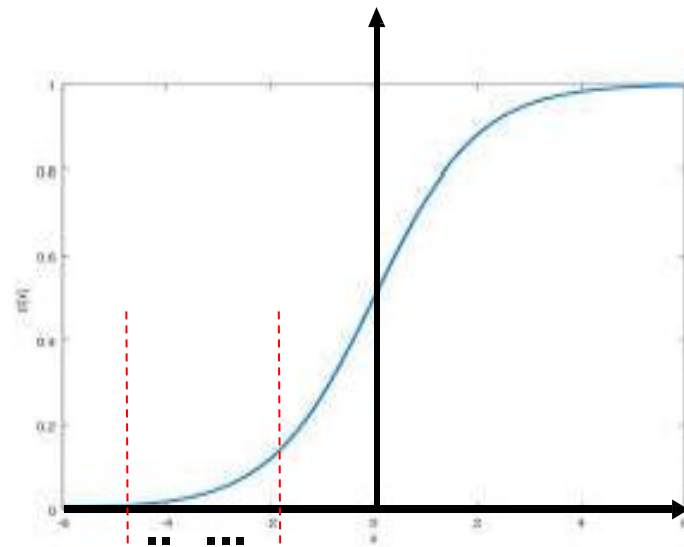
如果每一层的每个神经元进行批归一化，就能解决前向传递过程中的信号消失问题



批归一化与梯度消失

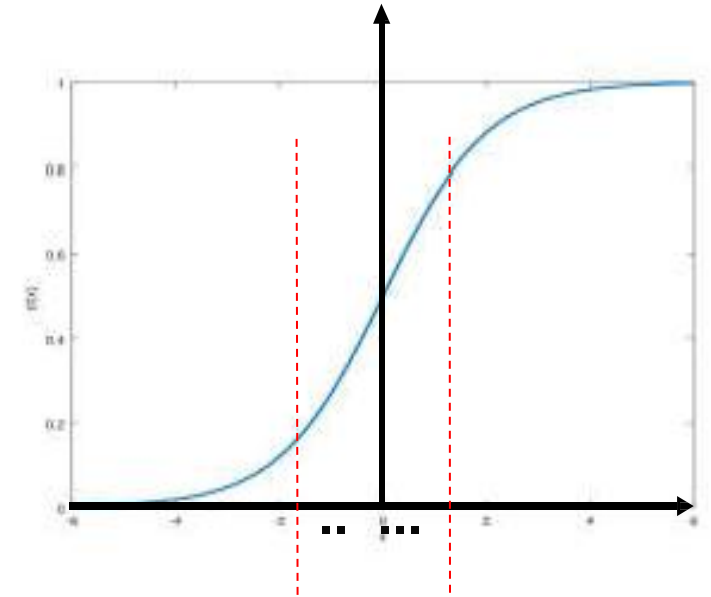


经常插入到全连接层后，非线性激活前



小梯度、
无梯度区域

减均值
除方差
→



批归一化



批归一化算法

输入: $B = \{x_1, \dots, x_m\}$;

学习参数: γ, β

输出: $\{y_1, \dots, y_m\}$

1. 计算小批量均值: $\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$
2. 计算小批量方差: $\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$
3. 归一化: $\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$
4. 平移缩放: $y_i \leftarrow \gamma \hat{x}_i + \beta$

问题: 输出的0均值1方差的正态分布是最有利于网络分类的分布吗?

回答: 根据对分类的贡献自行决定数据分布的均值与方差。

问题: 单张样本测试时, 均值和方差怎么设置?

回答: 来自于训练中。累加训练时每个批次的均值和方差, 最后进行平均, 用平均后的结果作为预测时的均值和方差。



全连接神经网络设计与学习



输入图像 X

图像表示

图像表示

分类模型

- 1、多层感知器
- 2、激活函数

模型参数

预测值 $\hat{Y}$

真实值 Y

优化算法

损失函数

- 3、SOFTMAX与交叉熵
- 4、对比多类支撑向量机损失

损失值

训练过程

- 8、权值初始化
- 9、批归一化
- 10、欠拟合、过拟合与Dropout
- 11、模型正则与超参数调优

- 5、计算图与反向传播
- 6、再谈激活函数
- 7、动量法与自适应梯度

参数更新



过拟合现象



出现过拟合，得到的模型在训练集上的准确率很高，但在真实的场景中识别率确很低。



过拟合与欠拟合

过拟合——是指学习时选择的模型所包含的参数过多，以至于出现这一模型对已知数据预测的很好，但对未知数据预测得很差的现象。这种情况下模型可能只是记住了训练集数据，而不是学习到了数据特征。

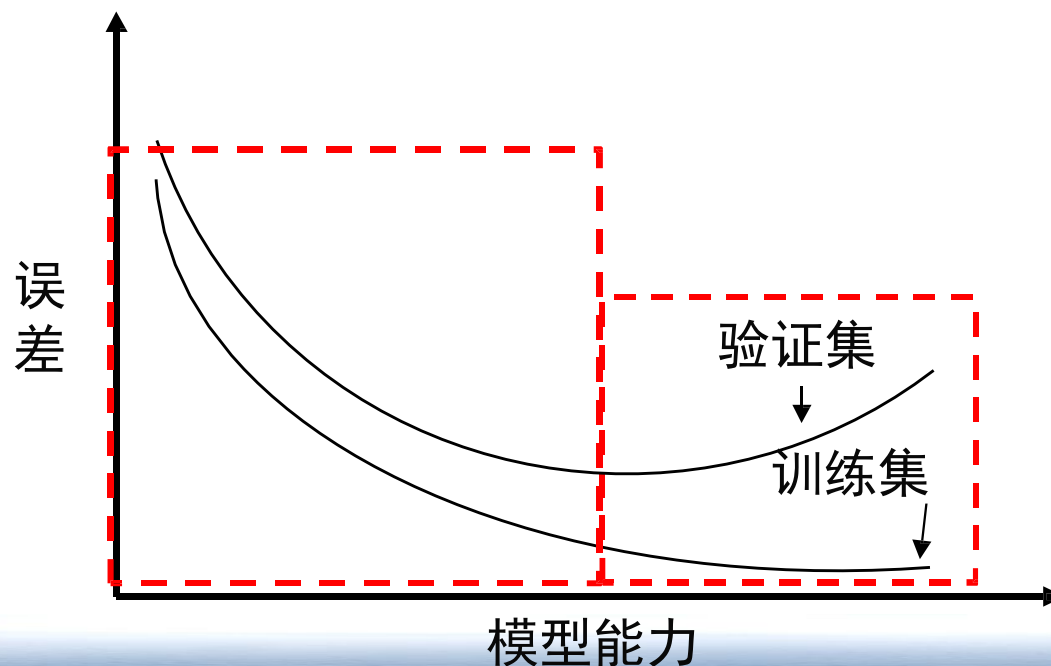
欠拟合——模型描述能力太弱，以至于不能很好地学习到数据中的规律。产生欠拟合的原因通常是模型过于简单。



学习过程中的过拟合

- ❖ 机器学习的根本问题是优化和泛化的问题。
- ❖ **优化**——是指调节模型以在训练数据上得到最佳性能；
- ❖ **泛化**——是指训练好的模型在前所未见的的数据上的性能好坏。

训练初期：优化和泛化是相关的：训练集上的误差越小，验证集上的误差也越小，**模型的泛化能力逐渐增强**



训练后期：模型在验证集上的错误率不再降低，转而开始变高。**模型**出现过拟合，开始**学习仅和训练数据有关的模式**。

应对过拟合

- ❖ **最优方案**——获取更多的训练数据
- ❖ **次优方案**——调节模型允许存储的信息量或者对模型允许存储的信息加以约束，该类方法也称为正则化。
 - 调节模型大小
 - 约束模型权重，即权重正则化（常用的有L1、L2正则化）



L2正则化

$$L(W) = \underbrace{\frac{1}{N} \sum_i L_i(f(x_i, W), y_i)}_{\text{数据损失}} + \underbrace{\lambda R(W)}_{\text{权重正则损失}}$$

L2正则损失: $R(W) = \sum_k \sum_l W_{k,l}^2$

L2正则损失对于大数值的权值向量进行严厉惩罚，鼓励更加分散的权重向量，使模型倾向于使用所有输入特征做决策，此时的模型泛化性能好！



应对过拟合

- ❖ **最优方案**——获取更多的训练数据
- ❖ **次优方案**——调节模型允许存储的信息量或者对模型允许存储的信息加以约束，该类方法也称为正则化。
 - 调节模型大小
 - 约束模型权重，即权重正则化（常用的有L1、L2正则化）
 - 随机失活（Dropout）

随机失活 (Dropout)

- ❖ 随机失活：让隐层的神经元以一定的概率不被激活。
- ❖ 实现方式：训练过程中，对某一层使用随机失活，随机将该层的一些输出舍弃(输出值设置为0)，这些被舍弃的神经元就好像被网络删除了一样。
- ❖ 随机失活比率 (Dropout ratio)：是指被设为0的特征所占的比例，通常在0.2-0.5范围内

例：假设某一层对给定输入样本的返回值应该是向量：[0.2, 0.5, 1.3, 0.8, 1.1]。

使用Dropout后，这个向量会有几个随机的元素变成：[0, 0.5, 1.3, 0, 1.1]。

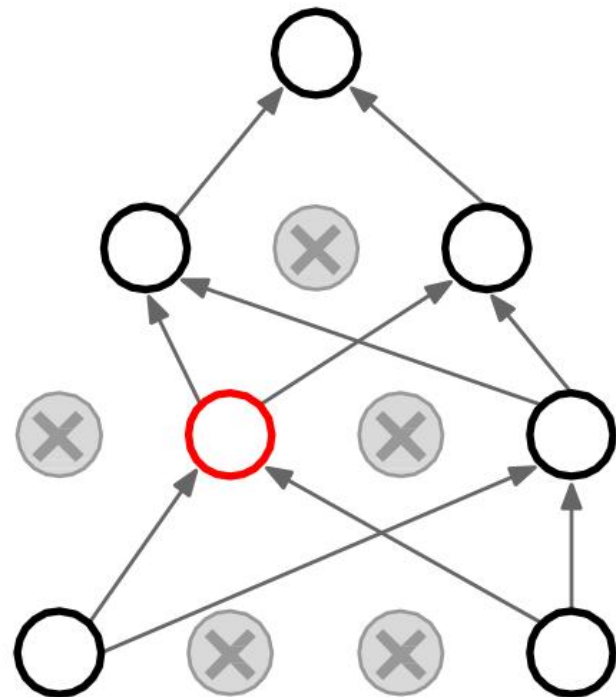
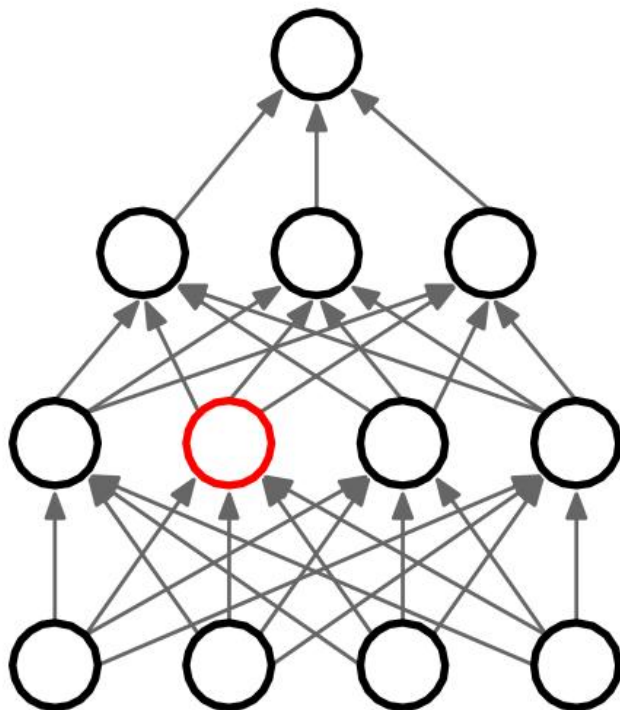
随机失活 (Dropout)

❖ 随机失活为什么能够防止过拟合呢？

解释1：随机失活使得每次更新梯度时参与计算的网路参数减少了，降低了模型容量，所以能防止过拟合。

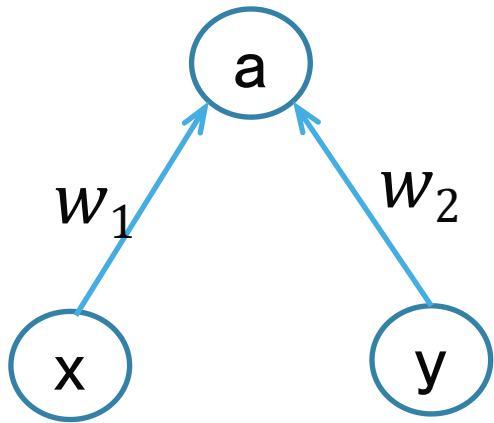
解释2：随机失活鼓励权重分散，从这个角度来看随机失活也能起到正则化的作用，进而防止过拟合。

解释3：随机失活 (Dropout) 可以看作模型集成。



随机失活的应用

存在的问题：



测试阶段： $E[a] = w_1x + w_2y$

训练阶段：
$$E[a] = \frac{1}{4}(w_1x + w_2y) + \frac{1}{4}(w_1x + 0y) + \frac{1}{4}(0x + w_2y) + \frac{1}{4}(0x + 0y)$$
$$= \frac{1}{2}(w_1x + w_2y)$$



随机失活的应用

三层神经网络示例：

```
1  p = 0.5 #神经元保持激活状态的概率，该值越高失活的单元就越少
2  def train(X):
3      H1 = np.maximum(0, np.dot(W1,X) + b1)
4      U1 = np.random.rand(*H1.shape) < p #第一层的mask
5      H1 *= U1 #第一层dropout操作
6      H2 = np.maximun(0, np.dot(W2,H1) + b2)
7      U2 = np.random.rand(*H2.shape) < p #第二层的mask
8      H2 *= U2 #第二层dropout操作
9      out = np.dot(W3,H2) + b3
10
11  def predict(X):
12      H1 = np.maximum(0, np.dot(W1,X) + b1) * p
13      H2 = np.maximun(0, np.dot(W2,H1) + b2) * p
14      out = np.dot(W3,H2) + b3
```



随机失活的应用

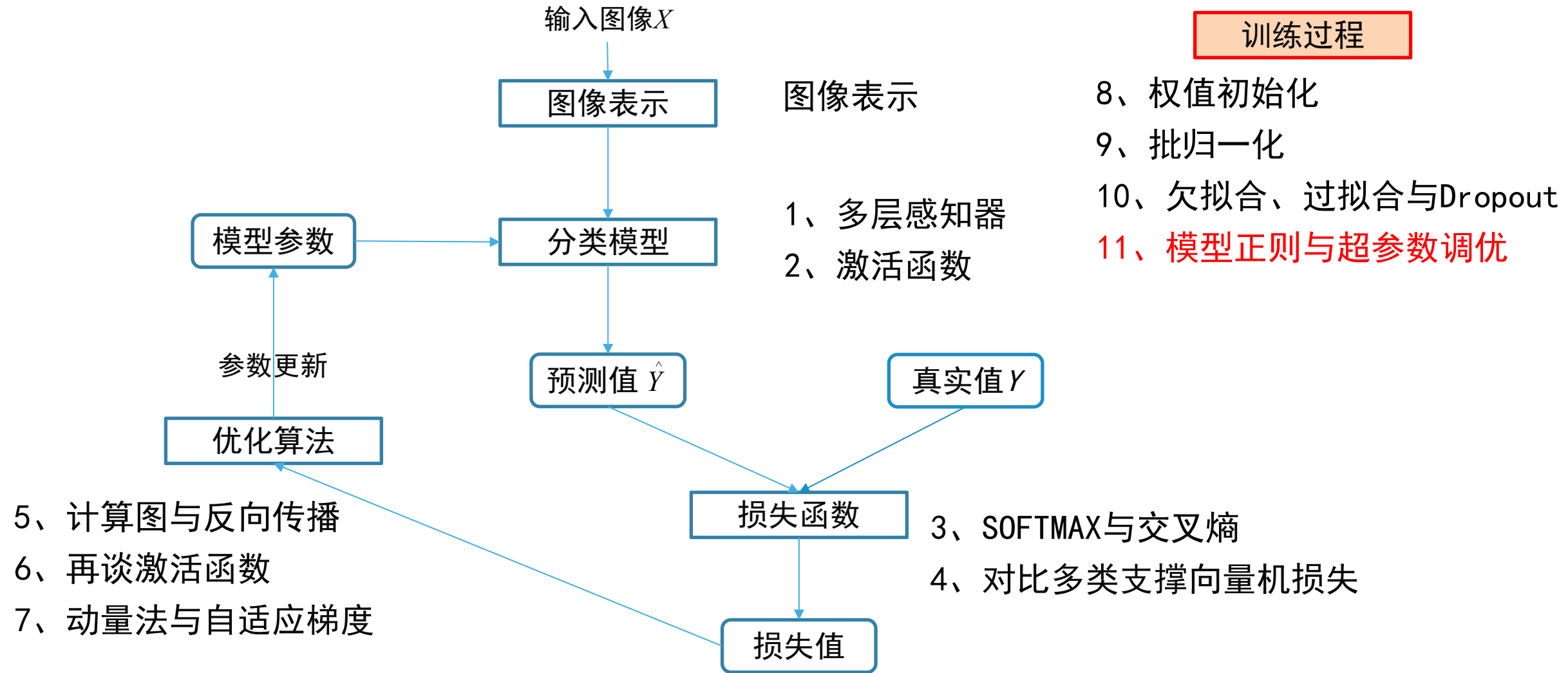
三层神经网络示例：

```
p = 0.5 #神经元保持激活状态的概率，该值越高失活的单元就越少
def train_new(X):
    H1 = np.maximum(0, np.dot(W1,X) + b1)
    U1 = (np.random.rand(*H1.shape) < p) / p #第一层的mask, 注意除了p
    H1 *= U1
    H2 = np.maximun(0, np.dot(W2,H1) + b2)
    U2 = (np.random.rand(*H2.shape) < p) / p #第二层的mask, 注意除了p
    H2 *= U2
    out = np.dot(W3,H2) + b3

def predict_new(X):
    H1 = np.maximum(0, np.dot(W1,X) + b1)
    H2 = np.maximun(0, np.dot(W2,H1) + b2)
    out = np.dot(W3,H2) + b3
```



全连接神经网络设计与学习



神经网络中的超参数

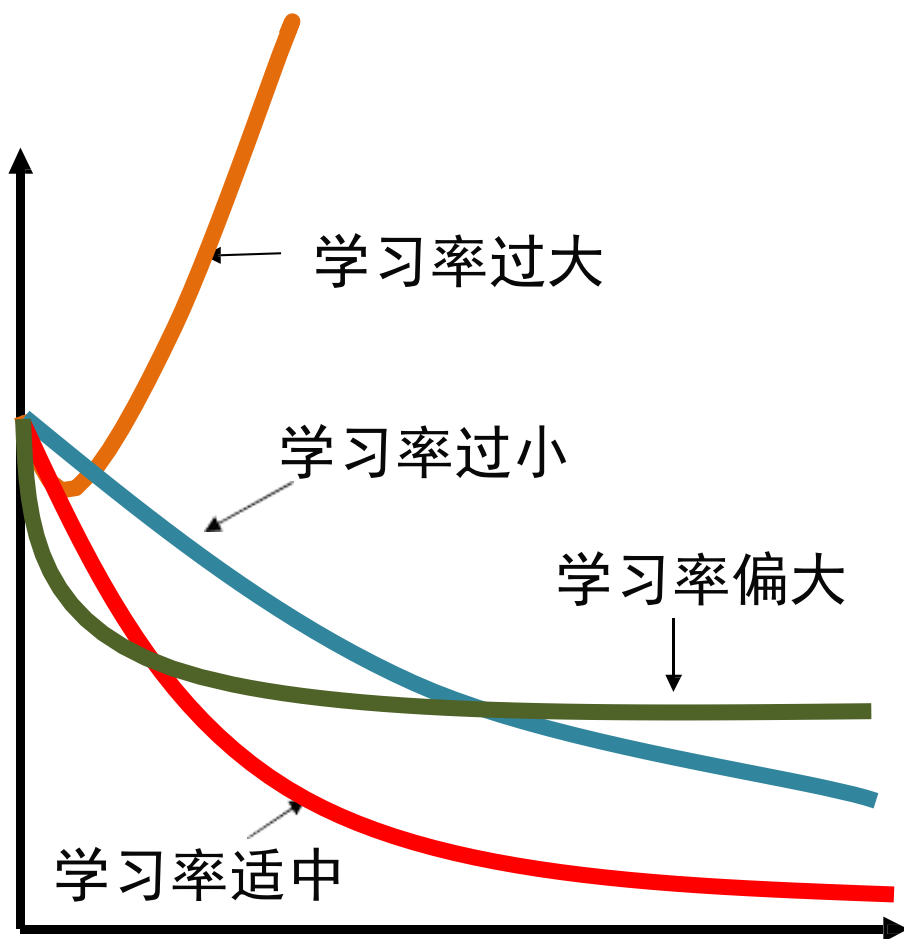
超参数：

- 网络结构——隐层神经元个数，网络层数，非线性单元选择等
- 优化相关——学习率、dropout比率、正则项强度等

超参数的重要性，如何找到合适的超参数？



学习率设置



- 学习率过大，训练过程无法收敛
- 学习率偏大，在最小值附近震荡，达不到最优
- 学习率过小，收敛时间较长
- 学习率适中，收敛快、结果好



超参数优化方法

网格搜索法：

- ① 每个超参数分别取几个值，组合这些超参数值，形成多组超参数；
- ② 在验证集上评估每组超参数的模型性能；
- ③ 选择性能最优的模型所采用的那组值作为最终的超参数的值。

随机搜索法：

- ① 参数空间内随机取点，每个点对应一组超参数；
- ② 在验证集上评估每组超参数的模型性能；
- ③ 选择性能最优的模型所采用的那组值作为最终的超参数的值。

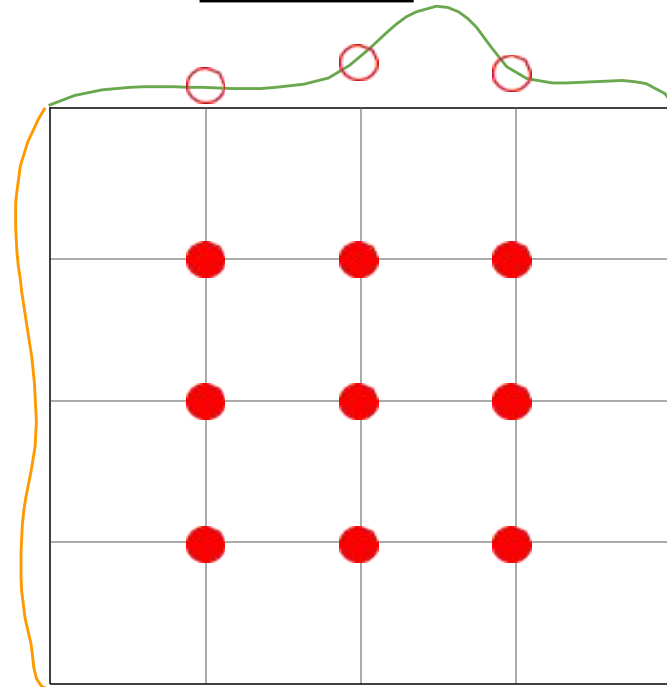


超参数优化方法

预算：9组实验

目标：最好的超参数组合

网格搜索



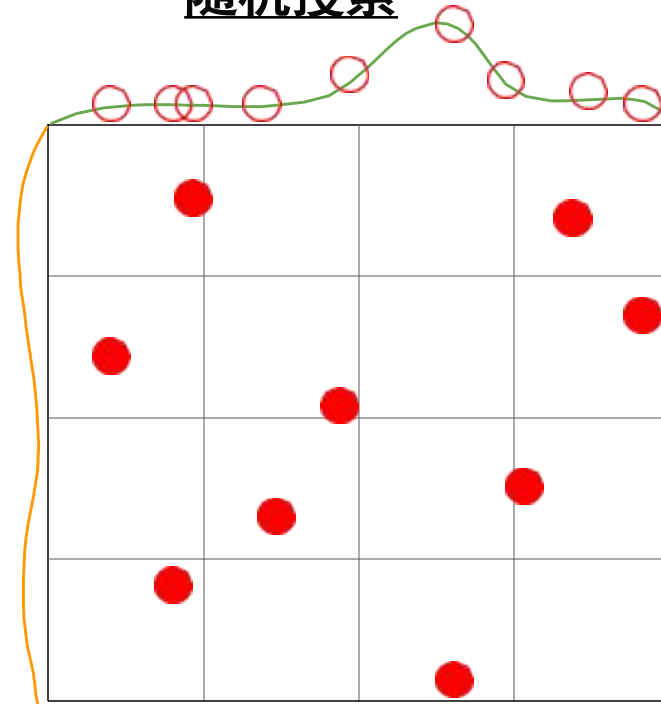
0.1 0.01 0.001

学习率
(重要参数)

δ (不重要参数)

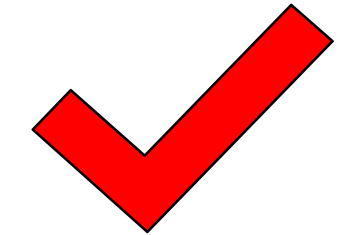
防止分母为零的参数,
 $\delta(10^{-5}, 10^{-6}, 10^{-7})$

随机搜索



0.1 0.01 0.001

学习率
(重要参数)



δ (不重要参数)

尝试了九
个不同的
学习率!

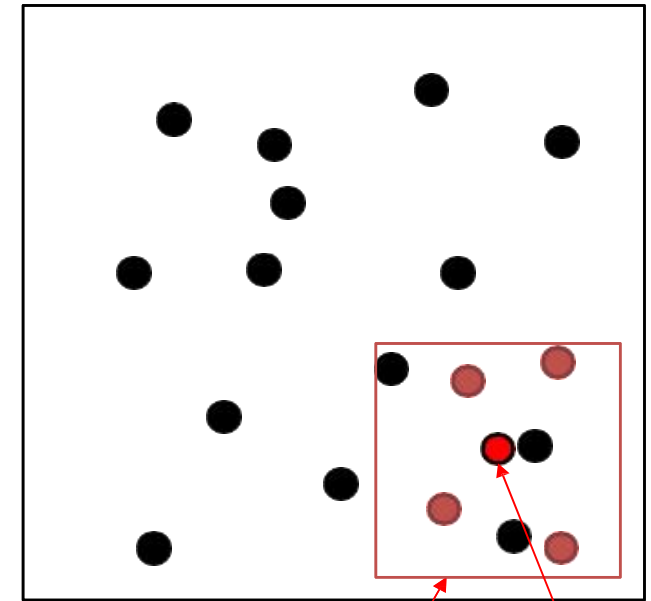
尝试了三
个不同的
学习率

超参数搜索策略

超参数搜索策略：

- **粗搜索：** 利用随机法在较大范围里采样超参数，训练一个周期，依据验证集正确率缩小超参数范围。
- **精搜索：** 利用随机法在前述缩小的范围内采样超参数，运行模型五到十个周期，选择验证集上精度最高的那组超参数。

参数空间



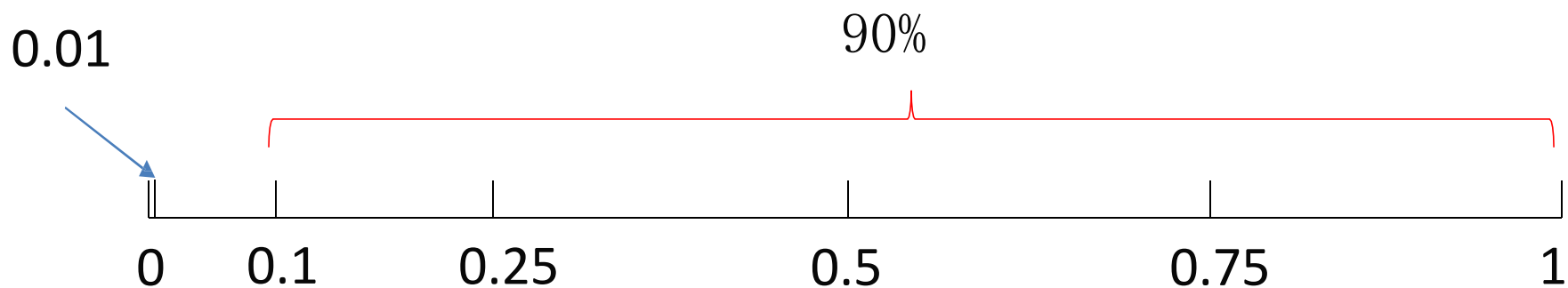
缩小后的
参数空间

最优超
参数



超参数的标尺空间

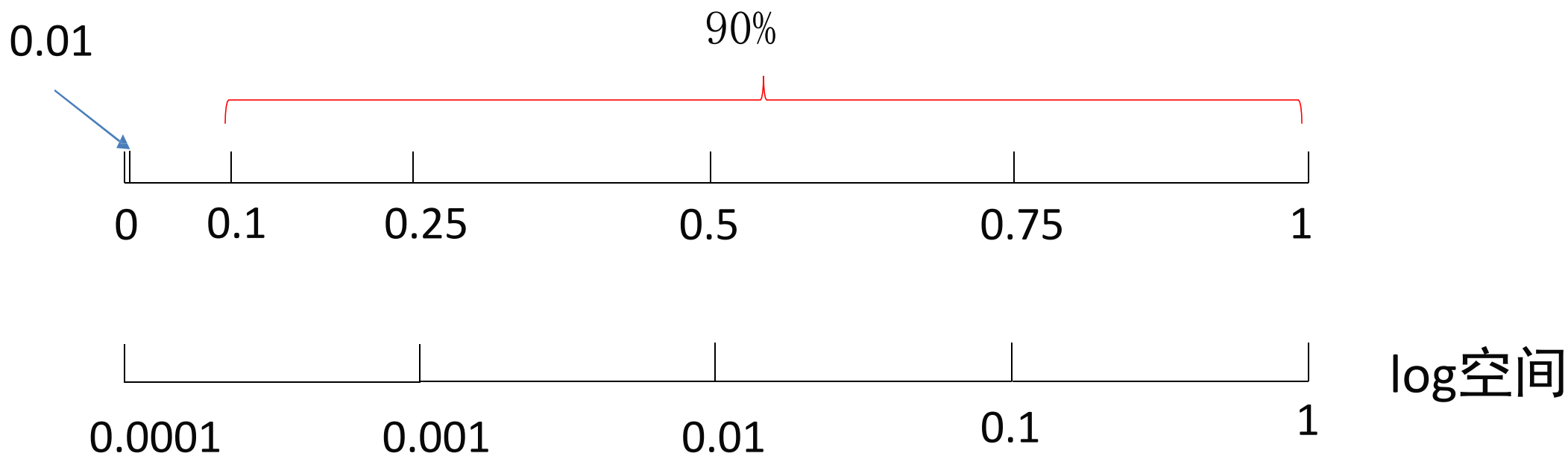
以学习率为例：



最优的值在 $[0.0001, 1]$ 之间，我们如何采样？

超参数的标尺空间

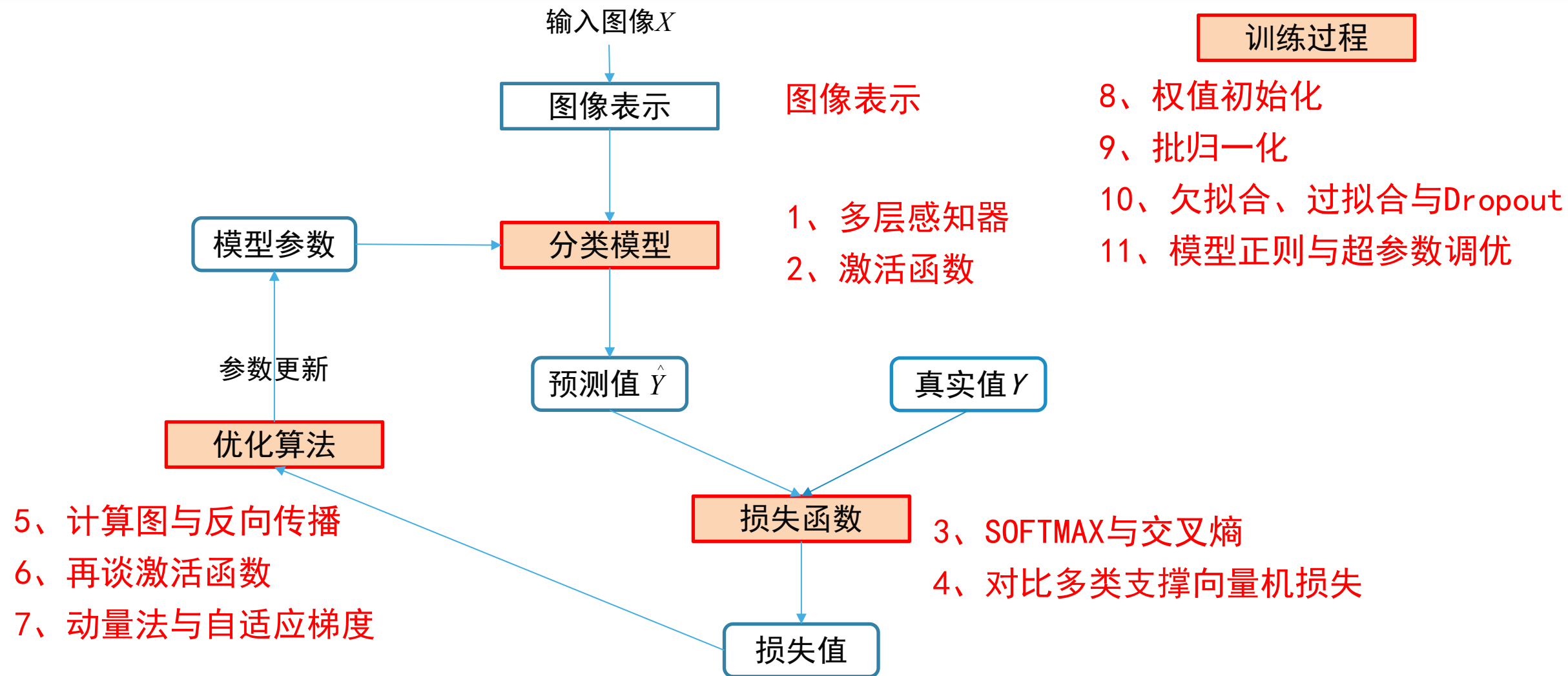
以学习率为例：



建议：对于学习率、正则项强度这类超参数，在对数空间上进行随机采样更合适！



全连接神经网络设计与学习



Q&A

联系方式: qbsun@bupt.edu.cn